

University of Brasília

Institute of Exact Sciences
Department of Computer Science

TransferKAN Under a Leak-Free Protocol: Re-evaluating Kolmogorov-Arnold Heads for Gunshot Wound Classification

Arthur Silva Carneiro

Thesis submitted as a partial requirement
for completion of the Bachelor of Science in Computer Science

Advisor
Prof. Dr. Pedro Garcia Freitas

Brasília
2026

Dedication

This work is dedicated to the individuals whose presence, sacrifice, and unwavering support made my academic journey possible. I dedicate this work to all of you, with sincere appreciation and respect.

Acknowledgements

I would like to express my sincere gratitude to the University of Brasília for granting me the opportunity to pursue my dream degree in Computer Science. It has been a privilege to study and develop academically at such a distinguished institution.

I also extend my thanks to FAPDF (Fundação de Apoio à Pesquisa do Distrito Federal) for the research scholarship provided during the first half of this project. This support was essential for the initial development of my work.

My appreciation goes to the GPDS (Grupo de Processamento Digital de Sinais) for providing the computational resources that enabled me to carry out significant portions of this research. I am equally grateful to LISA (Laboratório de Imagens, Sinais e Acústica) for granting additional access to computer infrastructure, which was fundamental for the completion of this study.

Finally, I would like to acknowledge the invaluable guidance of my advisor, Pedro Garcia Freitas. Beyond teaching me and supervising my research, he served as a mentor who offered meaningful advice regarding my academic and professional path. His support was instrumental to the successful development of this work.

This work was carried out with support from the Coordination for the Improvement of Higher Education Personnel - Brazil (CAPES), through access to the Portal of Journals.

Resumo

Classificar ferimentos por arma de fogo como de entrada ou de saída e estimar a distância do disparo são tarefas periciais exigentes. Este trabalho avalia a TransferKAN, uma ResNet-152 pré-treinada no ImageNet com uma Rede de Kolmogorov-Arnold (KAN) como cabeça de classificação, em fotografias reais de locais de crime da FDCPUnBGunshotDB. Uma versão anterior deste trabalho relatou que a TransferKAN superava os baselines de Lira et al., mas essa comparação misturava partições, inicializações e definições de AUC diferentes. Por isso, a avaliação foi refeita num protocolo único: cinco dobras agrupadas por caso, a mesma receita de treino para todos os modelos, AUC das probabilidades e regras de decisão fixadas antes dos experimentos.

Há quatro achados. (i) O hold-out do código público do estudo de 2024 estratifica pelo caso em vez de agrupar por caso: 98,7% das imagens de teste de entrada/saída pertencem a casos vistos no treino. Com a mesma rede e a mesma receita, ele infla a AUC em +0,034 em entrada/saída e de +0,155 a +0,158 na distância do disparo. (ii) A cabeça KAN não é a fonte dos ganhos: ela é equivalente às cabeças linear e MLP em entrada/saída (AUC de 0,905 contra 0,906) e fica abaixo de uma MLP do mesmo tamanho na distância do disparo, e uma KAN com wavelets não muda isso. (iii) Entre doze arquiteturas retreinadas, a Swin-B é a melhor (AUC de 0,929 em entrada/saída) e supera a TransferKAN nas duas tarefas. (iv) Uma única rede com uma cabeça por tarefa tem 47–50% menos parâmetros e é duas vezes mais rápida que o par de redes. Ela é não inferior na distância do disparo e cerca de 0,01 de AUC pior em entrada/saída, dentro da margem pré-registrada. Os disparos encostados continuam sendo a principal limitação. As contribuições são uma partição auditada, um protocolo sem vazamento e uma rede multitarefa a metade do custo.

Palavras-chave: Ferimentos por Arma de Fogo, Vazamento de Dados, Validação Cruzada Agrupada, Redes de Kolmogorov-Arnold (KAN), Aprendizado por Transferência, Aprendizado Multitarefa

Abstract

Classifying gunshot wounds as entry or exit and estimating the shooting distance are demanding forensic tasks. This work evaluates TransferKAN, a ResNet-152 pre-trained on ImageNet with a Kolmogorov-Arnold Network (KAN) as its classification head, on real crime-scene photographs of the FDCPUnBGunshotDB. An earlier version of this work reported that TransferKAN outperformed the baselines of Lira et al., but that comparison mixed different splits, initialisations and definitions of the AUC. The evaluation was therefore rebuilt under a single protocol: five folds grouped by case, the same training recipe for every model, AUC from probabilities and decision rules fixed before the experiments.

Four findings follow. (i) The hold-out of the public code of the 2024 study stratifies by case instead of grouping by case: 98.7% of its entry/exit test images belong to cases seen in training. With the same network and recipe, it inflates the AUC by +0.034 in entry/exit and by +0.155 to +0.158 in shooting distance. (ii) The KAN head is not the source of the gains: it is equivalent to linear and MLP heads in entry/exit (AUC 0.905 against 0.906) and below an MLP of the same size in shooting distance, and a wavelet KAN does not change this. (iii) Among twelve retrained architectures, Swin-B is the best (AUC 0.929 in entry/exit) and outperforms TransferKAN in both tasks. (iv) A single network with one head per task has 47–50% fewer parameters and is twice as fast as the pair of networks. It is non-inferior in shooting distance and about 0.01 of AUC worse in entry/exit, within the pre-registered margin. Contact shots remain the main limitation. The contributions are an audited split, a leak-free protocol and a multitask network at half the cost.

Keywords: Gunshot Wounds, Data Leakage, Grouped Cross-Validation, Kolmogorov-Arnold Networks (KAN), Transfer Learning, Multitask Learning

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Research Questions 2

1.2.1 The Original Hypothesis 2

1.3 Contributions 2

1.4 Organisation of This Document 3

2 Background **4**

2.1 Forensic Pathology of Gunshot Wounds 4

2.1.1 Entry versus Exit Wounds 4

2.1.2 Medico-Legal Shooting Distance (MLSD) 4

2.2 Deep Learning for Computer Vision 5

2.2.1 Convolutional Networks, ResNet and Transfer Learning 5

2.2.2 Vision Transformers 5

2.3 Kolmogorov-Arnold Networks 5

2.3.1 From the Representation Theorem to KAN Layers 5

2.3.2 B-Spline Edges and Parameter Count 6

2.3.3 Wavelet KANs 6

2.4 Evaluation Without Leakage 7

2.5 AUC from Probabilities and from Classes 7

2.6 Multitask Learning 8

2.7 Paired Comparisons and Non-Inferiority 8

3 Related Work **9**

3.1 AI in Forensic Pathology 9

3.2 The Study of Lira et al. 9

3.2.1 What the Article and the Public Code Say About the Split 10

3.3 KANs as Classification Heads 10

3.4 Research Gap 11

4 TransferKAN and the Experimental Design **12**
4.1 The TransferKAN Architecture . . . 12
4.2 Hypotheses Under Test . . . 13
4.3 Experimental Design . . . 13

5 Methodology **15**
5.1 Dataset . . . 15
5.2 Case-Grouped Folds . . . 16
5.3 Training Recipe . . . 18
5.4 Heads Compared on the ResNet-152 . . . 19
5.5 Backbones Under the Same Protocol . . . 19
5.6 Shooting Distance . . . 20
5.7 Reproducing the Split of Lira et al. . . . 20
5.8 Wavelet-KAN Heads . . . 20
5.9 One Network for Both Tasks . . . 21
5.10 Metrics . . . 21
5.11 Statistics and Pre-Registered Decision Rules . . . 22
5.12 Threats to Validity . . . 23

6 Results **24**
6.1 The 2024 Hold-Out Leaks Cases . . . 24
6.2 Entry/Exit: Heads and Backbones . . . 26
6.3 Shooting Distance: Heads and Backbones . . . 28
6.4 Comparison with the Published Numbers of Lira et al. . . . 30
6.5 TransferKAN and the Transformers . . . 32
6.6 Wavelet-KAN Heads . . . 32
6.7 One Network for Both Tasks . . . 35
6.8 Summary . . . 37

7 Conclusions **39**
7.1 Answers to the Research Questions . . . 39
7.2 Limitations . . . 40
7.3 Future Work . . . 41

References **42**

Appendix **45**

A Results in the Format of Lira et al. **46**

B Exploratory Experiments **48**

List of Figures

4.1 TransferKAN. A ResNet-152 pre-trained on ImageNet extracts a 2,048-dimensional feature vector, which passes through Dropout ($p = 0.5$) and a two-layer KAN head $[2048, 128, K]$ with grid size 10 and cubic splines ($K = 2$ for entry/exit, $K = 3$ for the shooting distance). In the head ablation only the orange block changes. 13
4.2 The multitask variant: one ResNet-152 and one Dropout shared by two heads of the same type (KAN or MLP-128). The loss is $L = L_{\mathrm{E/S}} + L_{\mathrm{dist}}$, with exit wounds masked out of the distance term. 14

5.1 Frequency of age at the time of death in the FDCPUnBGunshotDB dataset, reported by selected cases (a) and by selected images (b). 17

6.1 AUC of the ResNet-152 with the linear head (a) and with the KAN head (d), seed 42, on each case-grouped fold (circles) and on each reproduced partition of Lira et al. (squares). Same network, recipe and seed; only the split changes. 25
6.2 Stability analysis of the ResNet-152 of Lira et al. over grouped k-folds, for wound type (a) and shooting distance (b); the leftmost (grey) box of each panel is accuracy. 27
6.3 Out-of-fold ROC curves of the four heads on the ResNet-152 (entry/exit; probabilities averaged over seeds 42–44; exit is the positive class). 28
6.4 Out-of-fold one-vs-rest ROC curves of the four heads on the ResNet-152 for the shooting distance (entries only; probabilities averaged over seeds 42–44). The legend gives each head's one-vs-rest AUC. 29
6.5 AUC (mean ± s.d. across folds) against the number of head parameters (log scale) for the linear and MLP heads, the spline KAN and the wavelet-KAN heads. W3 and W4 have the parameter count of W1 and are shifted horizontally only for legibility; W3–W5 use a single seed. 33

List of Tables

4.1 Classification heads compared on the same ResNet-152 backbone (58,143,808 parameters) and the same recipe. K = number of classes (2 for entry/exit, 3 for MLSD). Parameters of the head only; seeds used in each task. 14

5.1 Composition of the five test folds (`StratifiedGroupKFold`, seed 42, groups = case, stratified by the four-class label). Contact: images (cases). No case appears in more than one fold. 16

6.1 Effect of the data split with everything else fixed (ResNet-152, same recipe, seed 42). *Lira et al.*: the image-level hold-out of the public code of Lira et al. (iterative stratification on case number and label) reproduced with three partitions; mean ± s.d. over the partitions. *Grouped*: the five case-grouped folds of this work; mean ± s.d. over folds. Leak: share of test images whose case also has images in training. Δ = Lira − grouped. 25

6.2 Entry/exit classification under the single protocol (case-grouped stratified 5-fold, same two-phase recipe, last epoch). Mean ± s.d. across folds; each fold is the mean over the training seeds (column S). F1 is the macro average (macro precision and recall are in Appendix A); AUC is computed from predicted probabilities. Min: training minutes per fold on an NVIDIA L4. 26

6.3 Medico-legal shooting distance (contact, close range, distant; entry wounds only) under the same protocol as Table 6.2. Mean ± s.d. across folds; S = training seeds (Swin-B and MaxViT-T: seeds 42 and 43 only). AUC = one-vs-rest macro AUC from probabilities. Rec.: per-class recall (mean across folds); contact has 4–13 test images per fold and is descriptive only. 29

6.4 Entry/exit: numbers published by Lira et al. [2] (Table 4) next to the same backbones under the single protocol of this work. The two sides differ in three ways at once: the split (image-level hold-out in their public code × case-grouped 5-fold), the initialisation (mostly trained from scratch in their public code × ImageNet weights) and the AUC (from predicted classes, AUC_c × from probabilities, AUC_p). Only AUC_c is computed in the same way on both sides; the table compares evaluation packages, not models. M = macro; ours: mean ± s.d. across folds. 31
6.5 MLSD (entries only): numbers published by Lira et al. [2] (Table 5) next to the same backbones under the single protocol of this work. The two sides differ in three ways at once: the split (image-level hold-out in their public code × case-grouped 5-fold), the initialisation (mostly trained from scratch in their public code × ImageNet weights) and the AUC (from predicted classes, AUC_c × from probabilities, AUC_p). Only AUC_c is computed in the same way on both sides; the table compares evaluation packages, not models. M = macro; ours: mean ± s.d. across folds. In this task, predicting always "distant" already gives an accuracy of 0.854. 31
6.6 TransferKAN (ResNet-152 + spline KAN, seeds 42–44) against the two transformer backbones with a linear head, under the single protocol. ΔAUC on the pooled out-of-fold predictions (probabilities averaged over the seeds); 95% CI from a paired case-level bootstrap (2,000 resamples); wins = folds in which TransferKAN has the higher AUC. 32
6.7 Entry/exit: wavelet-KAN heads against the heads (a)–(d) on the same ResNet-152 and recipe. Mean ± s.d. across folds (each fold averaged over the S training seeds); F1 is macro; AUC_{OOF} pools the out-of-fold predictions. Min: training minutes per fold (NVIDIA L4). Head ms: forward + backward of the head alone on a batch of 32 random 2048-d inputs. 33
6.8 MLSD (entries only; AUC = one-vs-rest macro): wavelet-KAN heads against the heads (a)–(d) on the same ResNet-152 and recipe. Mean ± s.d. across folds (each fold averaged over the S training seeds); F1 is macro; AUC_{OOF} pools the out-of-fold predictions. Min: training minutes per fold (NVIDIA L4). Head ms: forward + backward of the head alone on a batch of 32 random 2048-d inputs. 34

6.9 Pre-registered contrasts of the wavelet-KAN heads (Δ = W − reference) on the pooled out-of-fold predictions (probabilities averaged over the common seeds); 95% CI from a paired case-level bootstrap (2,000 resamples). Non-inferiority margin: −0.02 on the lower CI bound of both ΔAUC and ΔF1. Superior: higher AUC in at least 4/5 folds and ΔAUC CI above zero. Inferior: higher AUC in at most 1/5 folds and ΔAUC CI below zero. Inconclusive: none of the three. Wins: folds in which W has the higher AUC. 35

6.10 One multitask ResNet-152 (shared backbone, one head per task, $L = L_{\mathrm{E/S}} + L_{\mathrm{dist}}$ with exits masked out of L_{dist}) against the pair of single-task networks of the head ablation, same folds and recipe, seeds 42–44 on both sides. Mean ± s.d. across folds; AUC from probabilities (distance: one-vs-rest macro, entries only); F1 is macro; 4-class F1: exit/contact/close/distant pipeline (secondary, descriptive). 36

6.11 Pre-registered contrasts of the multitask study (Δ = first − second; MT = multitask, ST = pair of single-task networks), on the pooled out-of-fold predictions (probabilities averaged over seeds 42–44); 95% CI from a paired case-level bootstrap (2,000 resamples). Non-inferiority margin: −0.02 on the lower CI bound of ΔAUC. Worse: higher AUC in at most 1/5 folds and ΔAUC CI below zero; "within margin": that CI is still above −0.02. Wins: folds in which the first system has the higher AUC (F1 for the 4-class rows). 36

6.12 Cost of obtaining both outputs on an NVIDIA L4 (AMP fp16). Training: minutes per fold (single-task = sum of the two networks; mean ± s.d. over the 15 units) and peak training memory. Inference: ms per image to obtain both outputs (one forward pass for the multitask network, two for the pair; 20 warm-up and 200 timed runs with CUDA events, mean ± s.d.) and peak inference memory at batch 32. Timing does not depend on the weights and was measured on freshly built models. 37

A.1 Entry/exit under the single protocol, in the format of Lira et al. [2]: M = macro, m = micro, W = weighted. Mean across folds (the standard deviations are in the result files of the repository); AUC from predicted probabilities. Seeds as in the main tables. 46

A.2 MLSD (entries only) under the single protocol, in the format of Lira et al. [2]: M = macro, m = micro, W = weighted. Mean across folds (the standard deviations are in the result files of the repository); AUC from predicted probabilities. Seeds as in the main tables. 47

B.1 Exploratory KAN architectures trained from scratch (no ImageNet pre-training) in an earlier phase of this work. They used a single 80/20 hold-out split by case, inputs of 64 or 128 px, batches of 8 and their own recipe, so they are *not* comparable with any number obtained under the single protocol. M = macro, m = micro, W = weighted. 49

Chapter 1

Introduction

1.1 Problem Statement

Firearms are among the instruments most used to commit homicides worldwide [1]. Gunshot wounds are therefore common in forensic work, at crime scenes and in autopsies, and forensic experts must determine their nature. Two questions are central: whether a wound is an entry or an exit wound, and, for entry wounds, from which distance the shot was fired, the medico-legal shooting distance (MLSD).

Under ideal conditions, features such as the shape and size of the wound, the abrasion ring and the margins make entry and exit relatively easy to tell apart. In practice these features may be partly absent, overlapping or altered by putrefaction, and the calibre, the ammunition, the angle, intermediate targets and the body region complicate the classification even for experienced forensic pathologists.

Deep learning has changed medical image analysis, but its use in forensic practice is still limited. One of the main obstacles is data: forensic photographs are taken with different equipment, under varied conditions and in the routine of crime scenes, which makes standardisation impractical and produces small, imbalanced datasets. Lira et al. [2] built the first large database of real crime-scene photographs of gunshot wounds, the FDCPUnBGunshotDB, and evaluated 59 deep learning architectures on it.

This work started from a specific proposal: TransferKAN, a ResNet-152 pre-trained on ImageNet whose dense classification head is replaced by a Kolmogorov-Arnold Network (KAN) [3]. An earlier version of this dissertation reported that TransferKAN outperformed the baselines of Lira et al. Re-examining that comparison showed that it mixed different things. The published baselines came from a split in which, according to the public code of the study, images of the same case appear in training and test, with most architectures trained from scratch and the AUC computed from predicted classes. TransferKAN, in turn, had been evaluated with case-grouped folds, a pre-trained backbone,

the AUC from probabilities and an epoch chosen on the test set. A difference between such numbers says little about the KAN head. This dissertation rebuilds the evaluation from scratch, under a single protocol, and asks what can actually be concluded.

1.2 Research Questions

The work is organised around five research questions:

- **RQ1:** Does the hold-out used in 2024 inflate the reported numbers, and by how much, when everything else is kept fixed?
- **RQ2:** Under a protocol without case leakage, does a KAN head improve over linear and MLP heads on the same pre-trained backbone?
- **RQ3:** How do twelve of the architectures evaluated by Lira et al. compare under the same protocol, and where does TransferKAN stand among them?
- **RQ4:** Does a KAN with a different basis, wavelets instead of splines, change the answer to RQ2?
- **RQ5:** Can a single network with one head per task replace the pair of networks currently used for entry/exit and shooting distance?

1.2.1 The Original Hypothesis

The hypothesis that motivated TransferKAN was that replacing the MLP head of a pre-trained ResNet-152 with a KAN would improve discrimination, measured by the AUC, in both tasks, because learnable edge functions would model the decision boundaries of imbalanced forensic classes better. This hypothesis is not discarded; it becomes RQ2 and is tested with a controlled ablation and a decision rule fixed before the experiments (Section 5.11). Its answer is negative.

1.3 Contributions

The contributions of this dissertation are methodological and practical:

- **An audit of the 2024 split.** The hold-out of the public code of Lira et al. stratifies by case instead of grouping by case. Reproducing it shows that 98.7% (entry/exit) and 98.4% (shooting distance) of the test images have their case in training, and that, with the same network and recipe, this split inflates the AUC by +0.034 in entry/exit and by +0.155 to +0.158 in shooting distance.

- **A single protocol without leakage.** Case-grouped folds, one training recipe for every model, AUC from probabilities, a frozen and hashed implementation and decision rules written before the runs. Under it, twelve architectures, five heads, five wavelet-KAN heads and a multitask network were trained and compared.
- **A controlled head ablation with spline and wavelet KANs.** The KAN head of TransferKAN is equivalent to linear and MLP heads in entry/exit and below an MLP of the same size in shooting distance. Wavelet KANs do not change this, and neither family is parameter-efficient or faster in this setting. The best architecture is Swin-B, which outperforms TransferKAN in both tasks.
- **One network for both tasks at half the cost.** A single ResNet-152 with two heads has about half the parameters of the pair of networks and obtains both outputs twice as fast. It is non-inferior in shooting distance and slightly but consistently worse in entry/exit, within the pre-registered margin.

The results also show where the difficulty lies: contact shots, with 42 images from 16 cases, are almost never recognised when their case is absent from training.

1.4 Organisation of This Document

Chapter 2 presents the forensic tasks, the deep learning components, KANs with spline and wavelet bases, and the evaluation concepts used throughout. Chapter 3 reviews related work, in particular the study of Lira et al. as described in the article and as implemented in its public code. Chapter 4 presents TransferKAN, the hypotheses it embodies and the experimental design. Chapter 5 describes the single protocol. Chapter 6 reports the results in the order of the research questions, and Chapter 7 answers them and discusses limitations and future work. Appendix A gives the results in the format of Lira et al., and Appendix B records the exploratory from-scratch KAN architectures.

Chapter 2

Background

This chapter gives the background needed for the rest of the dissertation: the forensic tasks, the deep learning components, the Kolmogorov-Arnold Networks with spline and wavelet bases, and the evaluation concepts on which the conclusions rest, namely grouped validation, the two ways of computing an AUC, multitask learning and paired comparisons.

2.1 Forensic Pathology of Gunshot Wounds

In forensic medicine, the analysis of gunshot wounds helps to reconstruct the events of a crime. A projectile produces morphological changes that depend on the firearm, the ammunition and the distance of the shot [4, 5].

2.1.1 Entry versus Exit Wounds

An entry wound typically shows an *abrasion ring* (a rim of scraped skin), inverted tissue and a regular, circular or oval shape that depends on the angle of incidence. An exit wound usually shows everted tissue, an irregular or stellate shape, and no abrasion ring or gunshot residue, because the projectile pushes the tissue outward from within. Intermediate targets, deflection on bone and post-mortem changes such as putrefaction can hide these signs, so the visual classification depends heavily on the experience of the expert.

2.1.2 Medico-Legal Shooting Distance (MLSD)

For entry wounds, the medico-legal shooting distance is estimated from the macroscopic traces left by the combustion of gunpowder and primer [4]:

- **Contact:** the muzzle touches the skin. The wound often shows a muzzle imprint and lacerations caused by the expanding gases.
- **Close range:** the shot is fired from a short distance (typically less than about 60 cm), and the wound shows *tattooing* or *stippling* (unburned powder grains embedded in the skin) and soot.
- **Distant:** powder residues no longer reach the skin, and the wound shows essentially the abrasion ring.

Exit wounds carry no distance information, which is why the distance task uses only entries.

2.2 Deep Learning for Computer Vision

2.2.1 Convolutional Networks, ResNet and Transfer Learning

Convolutional neural networks extract hierarchical features from images, from edges in the first layers to complex shapes in the deeper ones. Residual networks (ResNets) [6] add skip connections that ease the training of very deep models; the ResNet-152 has 152 layers and produces a 2,048-dimensional feature vector before its last layer. In transfer learning, a network pre-trained on a large dataset such as ImageNet [7] is reused as a feature extractor and fine-tuned on the target task. This is especially useful when the target dataset is small, as in forensic imaging. The last layer that maps the features to the classes is called the *head*; it is usually a linear layer or a small multi-layer perceptron (MLP).

2.2.2 Vision Transformers

Transformers process an image as a sequence of patches with self-attention. Two of the architectures evaluated here are of this kind: the Swin Transformer [8], which computes attention inside shifted local windows and builds a hierarchy of resolutions, and MaxViT [9], which combines convolutions with local and dilated global attention.

2.3 Kolmogorov-Arnold Networks

2.3.1 From the Representation Theorem to KAN Layers

The Kolmogorov-Arnold representation theorem states that any continuous multivariate function on a bounded domain can be written as a finite composition of continuous uni-

variate functions and additions. Kolmogorov-Arnold Networks (KANs) [3] turn this idea into a trainable layer. In an MLP, each edge carries a scalar weight and each node applies a fixed nonlinearity. In a KAN layer, each edge carries a learnable univariate function $\phi_{j,i}$, and the node only adds:

$$x_j^{(l+1)} = \sum_{i=1}^{n_l} \phi_{j,i}^{(l)}\left(x_i^{(l)}\right). \tag{2.1}$$

A KAN $[n_0, n_1, n_2]$ has two such layers; the head of TransferKAN is $[2048, 128, K]$.

2.3.2 B-Spline Edges and Parameter Count

In the original KAN and in the `efficient-kan` implementation [10] used here, each edge function is the sum of a base term and a spline:

$$\phi(x) = w_b \,\mathrm{SiLU}(x) + w_s \sum_{m=1}^{G+k} c_m B_m(x), \tag{2.2}$$

where the B_m are B-splines of order k on a grid of G intervals over $[-1, 1]$, extended by k intervals on each side. With $G = 10$ and $k = 3$ there are 13 coefficients c_m per edge, plus the weights w_b and w_s: 15 parameters per edge. Outside the extended grid the B-splines vanish and the edge reduces to its base term.

The parameter count matters for the claims of this work. The KAN paper argues that KANs can be more parameter-efficient than MLPs, mainly on small scientific and symbolic tasks [3]. As a classification head on 2,048 features, however, a KAN is large: the head $[2048, 128, 2]$ has $15 \times (2048 \times 128 + 128 \times 2) = 3{,}936{,}000$ parameters, whereas an MLP with the same widths has 262,530. Whether a KAN head is more efficient is therefore an empirical question, tested in this work against MLPs of the same size.

2.3.3 Wavelet KANs

Wav-KAN [11] replaces the spline by a wavelet with learnable scale s and translation τ on each edge:

$$\phi(x) = w_\psi \,\psi\!\left(\frac{x - \tau}{s}\right) + w_b \,\mathrm{SiLU}(x), \tag{2.3}$$

followed by batch normalisation of the layer output. This uses four parameters per edge instead of fifteen. Three mother wavelets are used here, written up to normalisation constants: the Mexican hat $\psi(t) = (1 - t^2)\, e^{-t^2/2}$, the derivative of Gaussian $\psi(t) = -t\, e^{-t^2/2}$ and the Morlet $\psi(t) = \cos(\omega_0 t)\, e^{-t^2/2}$ with $\omega_0 = 5$. The learnable scale and translation could, in principle, adapt each edge to the range of its input; whether they do is checked in Section 6.6.

2.4 Evaluation Without Leakage

Data leakage happens when information that would not be available at prediction time reaches the training of a model, so that its test performance overestimates its performance on new data [12]. A common form arises with grouped data: when several observations come from the same unit (a patient, a site, a case), observations of the same unit are more similar to each other than to the rest. If they are split across training and test, the model can recognise the unit instead of learning the task [13]. The remedy is grouped validation: every group is placed entirely in training or entirely in test.

In the gunshot database, a case often contributes several photographs of the same wound or of wounds of the same victim. The evaluation unit is therefore the case. Grouped k-fold cross-validation assigns whole cases to folds; stratified grouped k-fold does the same while keeping the class proportions similar across folds. Stratifying *by* the case identifier is different: iterative stratification [14] on the case number tries to put a proportional share of the images of each case in each subset, so it deliberately spreads every case over training and test. The difference between grouping and stratifying by case is at the centre of Section 6.1.

2.5 AUC from Probabilities and from Classes

The ROC curve plots the true positive rate against the false positive rate as the decision threshold on a continuous score varies, and the area under it (AUC) equals the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative one [15]. It measures how well the model ranks the examples, independently of the threshold. For more than two classes, the one-vs-rest macro AUC averages the AUC of each class against the others.

If the AUC is computed from predicted labels instead of scores, there is a single threshold and the ROC "curve" has one interior point. Its area is then

$$\mathrm{AUC}_c = \frac{\mathrm{TPR} + \mathrm{TNR}}{2}, \tag{2.4}$$

the average of sensitivity and specificity, which in the binary case equals the balanced accuracy and the macro recall. AUC_c depends on the threshold and is usually much lower than the AUC from probabilities of the same model. In this dissertation, AUC always means the AUC from probabilities; AUC_c is computed only to compare with the tables of Lira et al., who report it (Chapter 3).

2.6 Multitask Learning

Multitask learning trains one model on several related tasks so that they share a representation [16, 17]. In the most common form, hard parameter sharing, a single backbone feeds one head per task, and the loss is a weighted sum of the task losses. Sharing can help when the tasks use related features and can hurt when one task dominates the shared representation. When a label is undefined for some examples, as the shooting distance is for exit wounds, those examples are masked out of the corresponding loss term. A shared backbone also has a direct practical effect: both outputs come from one forward pass instead of two.

2.7 Paired Comparisons and Non-Inferiority

When two models are evaluated on the same test images, their difference should be estimated in a paired way, since both are affected by the same hard and easy images. A nonparametric bootstrap [18] resamples the evaluation units with replacement and recomputes the difference in each resample; the percentiles of the resampled differences give a confidence interval. Because images of the same case are dependent, the units resampled here are cases, not images.

A superiority question asks whether the interval of the difference lies above zero. A *non-inferiority* question asks something weaker: whether a model is not worse than a reference by more than a margin δ fixed in advance, that is, whether the lower bound of the interval of the difference lies above $-\delta$. Non-inferiority is the appropriate question when the candidate has another advantage, such as fewer parameters or a lower cost, and the margin must be set before the results are seen.

Chapter 3

Related Work

This chapter reviews the work on deep learning for gunshot wounds, describes in detail the study of Lira et al. [2], which provides the database and the baselines of this dissertation, and reviews the use of KANs as classification heads. It closes with the gap that motivates this work.

3.1 AI in Forensic Pathology

Artificial intelligence has entered forensic science more slowly than clinical medicine. Early studies of gunshot wounds used animal carcasses, such as pigs, to simulate human skin [19]. These controlled settings allowed high accuracies but lack the variability, contamination and morphological changes of real cases. Cheng et al. [20] trained classifiers on human autopsy photographs of entry and exit wounds, obtained under the relatively controlled conditions of necropsy, and did not address the shooting distance.

3.2 The Study of Lira et al.

Lira et al. [2] built the FDCPUnBGunshotDB, with 2,551 photographs (1,883 entry and 668 exit wounds) from 489 cases of the Federal District Civil Police, and evaluated 59 deep learning architectures from TorchVision on two tasks: entry versus exit and the medico-legal shooting distance (contact, close range, distant). Their hold-out tables (Tables 4 and 5 of the article) rank the ResNet-152 first in both tasks. In entry/exit it has accuracy 0.869 and macro AUC 0.821, the "86.90%" and "82.09%" of their abstract. In the distance task it has accuracy 0.925, macro AUC 0.733 and micro AUC 0.944; the "AUC up to 94.36%" of the abstract is the micro average. The article also reports a k-fold analysis of the ResNet-152 as boxplots (their Figure 4, reproduced in Figure 6.2); in entry/exit the median accuracy over the folds is about 0.70.

3.2.1 What the Article and the Public Code Say About the Split

The article describes both evaluations as split by case: images of a case used in training would not be used for testing, and the k-fold would use stratified grouped folds with the crime cases as groups [2]. The public code of the study (`leguwoi` [21], commit `83d82f1`) agrees for the k-fold, which uses `StratifiedGroupKFold` with the case number as group, but not for the hold-out. The function `iterative_split` applies iterative stratification [14] to the columns *case number* and *label*, that is, it *stratifies* by case instead of grouping by case. Stratifying on the case number spreads the images of each case over training and test. Reproducing that function on the distributed data, 98.7% of the entry/exit test images and 98.4% of the distance test images have their case in training (Section 6.1). It cannot be established whether the published tables were produced by exactly this version of the code; the statements of this dissertation about the hold-out refer to the public code.

Three other aspects of the public code matter for comparing numbers:

- **AUC from predicted classes.** The AUC is computed with `roc_auc_score` on the predicted labels, not on probabilities. With a single threshold, the macro AUC equals the average of recall and specificity, which in the binary case is the macro recall. This is visible in their Table 4, where the macro AUC of every model equals its macro recall (0.821 and 0.821 for the ResNet-152).
- **Initialisation.** Only the ResNets start from ImageNet weights, and their first convolution is replaced by a randomly initialised layer; the other families are trained from scratch.
- **Case identity.** Cases are identified by their number without the year, which merges cases from different years (417 groups instead of 486).

These observations do not reduce the value of the database or of the study, which is the first to classify gunshot wounds from real crime scenes at this scale. They do mean that the numbers of Tables 4 and 5 measure a different quantity from the case-grouped evaluation adopted here, and the earlier version of this dissertation compared the two as if they were the same.

3.3 KANs as Classification Heads

Kolmogorov-Arnold Networks [3] place learnable univariate functions on the edges of the network instead of fixed activations on the nodes (Section 2.3). The original article argues for accuracy and parameter efficiency mainly on small scientific and symbolic tasks. In

vision, KANs have been used both inside convolutions, as in the convolutional KANs of `torch-conv-kan` [22], and as heads on top of pre-trained extractors, as in TransferKAN. Several bases have been proposed for the edge functions, among them the B-splines of the original KAN, implemented efficiently in `efficient-kan` [10], and wavelets in Wav-KAN [11]. When a KAN head is compared with a dense head, the comparison often changes the backbone, the training recipe or the number of parameters at the same time, which makes it hard to attribute a difference to the head itself.

3.4 Research Gap

Three gaps motivate this work. First, the numbers available for this database come from a hold-out in which, according to the public code, images of the same case appear in training and test, with most architectures trained from scratch and the AUC computed from classes. No study had evaluated the architectures under a common protocol without case leakage. Second, the claim that a KAN head improves over a dense head on the same pre-trained backbone had not been tested with a controlled ablation that keeps the backbone, the recipe and the parameter budget fixed. Third, the two forensic tasks are learned by separate networks, although they share the same images. This dissertation addresses the three gaps with a single protocol (Chapter 5).

Chapter 4

TransferKAN and the Experimental Design

This chapter presents TransferKAN, the model this work set out to evaluate, the hypotheses that motivated it, and the experimental design used to test them. The design isolates the contribution of each component: the head, the backbone, the split and the number of tasks.

4.1 The TransferKAN Architecture

TransferKAN[1] combines a convolutional feature extractor pre-trained on ImageNet with a Kolmogorov-Arnold Network (KAN) as the classification head. It has three stages (Figure 4.1):

1. **Feature extractor:** a ResNet-152 [6] with ImageNet weights [7] and without its last fully connected layer. For a 224×224 image it outputs a vector of 2,048 features.
2. **Regularisation:** Dropout with $p = 0.5$ on that vector.
3. **KAN head:** a two-layer KAN, $[2048, 128, K]$, with $K = 2$ outputs for entry/exit and $K = 3$ for the shooting distance. Each edge carries a learnable function made of a SiLU base term and a cubic B-spline on a grid of size 10 (Section 2.3), implemented with `efficient-kan` [10]. The head has 3,936,000 parameters for entry/exit and 3,937,920 for distance.

[1] The short paper derived from this work for SIBGRAPI 2026 calls the same model TransKAN.

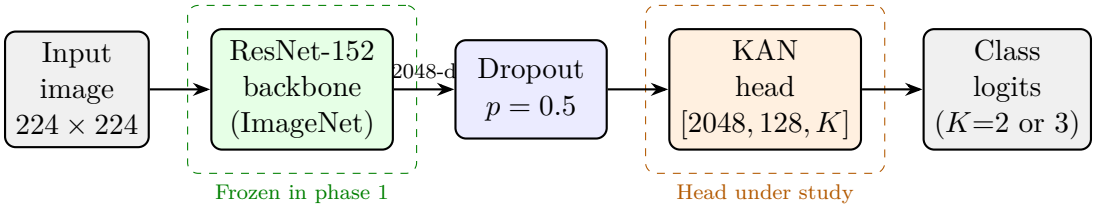


Figure 4.1: TransferKAN. A ResNet-152 pre-trained on ImageNet extracts a 2,048-dimensional feature vector, which passes through Dropout ($p = 0.5$) and a two-layer KAN head $[2048, 128, K]$ with grid size 10 and cubic splines ($K = 2$ for entry/exit, $K = 3$ for the shooting distance). In the head ablation only the orange block changes.

4.2 Hypotheses Under Test

TransferKAN was motivated by two claims about KANs used as classification heads. In the earlier version of this work they were stated as advantages; here they are treated as hypotheses, each with the result that would support it:

- **H1, better discrimination:** because its edge functions are learned, a KAN head would draw finer decision boundaries than an MLP head on the same features, especially for minority classes. This would be supported by a higher AUC than the best MLP head on the same backbone, under the pre-registered rule of Section 5.11 (outcome A).
- **H2, parameter efficiency:** a KAN head would reach the same performance with fewer parameters. This would be supported by a KAN head beating an MLP head with the same number of parameters. Note that the TransferKAN head is not small: with 15 parameters per edge it has 3.94 million parameters, 15 times as many as an MLP with the same widths (262,530).

4.3 Experimental Design

Head ablation. To test H1 and H2, the ResNet-152, the folds and the recipe are kept fixed and only the head changes (Table 4.1). The linear head (a) is the simplest reference. The MLP-128 (b) has the same widths as the KAN. The matched MLP (c) has practically the same number of parameters as the KAN, so that a difference between (c) and (d) cannot be attributed to size. The KAN-8 (e) gives a KAN the parameter budget of (b). To check whether the conclusion depends on the spline basis, wavelet-KAN heads [11] W1–W5 replace the B-splines by wavelets: W1 keeps the topology of (d), W2 the budget of (b) and W5 the budget of (d), and W3 and W4 change the wavelet.

Backbones. Twelve architectures of the study of Lira et al. [2], among them the ResNet-152 and two transformers, are retrained with the same recipe and a linear head.

Table 4.1: Classification heads compared on the same ResNet-152 backbone (58,143,808 parameters) and the same recipe. K = number of classes (2 for entry/exit, 3 for MLSD). Parameters of the head only; seeds used in each task.

			Head parameters		Seeds		
Id	Head	Structure	E/S	MLSD	E/S	MLSD	Role
(a)	Linear	Linear$(2048, K)$	4,098	6,147	42–44	42–44	reference
(b)	MLP-128	$[2048, 128, K]$, ReLU	262,530	262,659	42–44	42–44	reference
(c)	MLP, matched to (d)	$[2048, 1919, K]$, ReLU	3,935,871	3,937,791	42–44	42–44	same budget as (d)
(d)	Spline KAN = TransferKAN	$[2048, 128, K]$, $G = 10$, $k = 3$	3,936,000	3,937,920	42–44	42–44	object of study
(e)	KAN-8	$[2048, 8, 2]$, $G = 10$, $k = 3$	246,000	—	42	—	sensitivity
W1	Wav-KAN, Mexican hat	$[2048, 128, K]$	1,049,860	1,050,374	42–44	42–44	topology of (d)
W2	Wav-KAN, Mexican hat	$[2048, 32, K]$	262,468	262,598	42–44	42–44	same budget as (b)
W3	Wav-KAN, DoG	$[2048, 128, 2]$	1,049,860	—	42	—	sensitivity
W4	Wav-KAN, Morlet	$[2048, 128, 2]$	1,049,860	—	42	—	sensitivity
W5	Wav-KAN, Mexican hat	$[2048, 480, 2]$	3,936,964	—	42	—	same budget as (d)

This shows where TransferKAN stands among architectures trained in the same way. Since every backbone starts from ImageNet weights, the effect of pre-training is not isolated (Section 6.4).

Split. The hold-out of the public code of Lira et al. is reproduced and compared with the case-grouped folds with everything else fixed, to measure how much the split alone changes the numbers.

One network for both tasks. Finally, a single ResNet-152 with two heads, one per task, is compared with the pair of single-task networks (Figure 4.2). This variant changes the number of networks and keeps the type of head, so it measures the cost and the performance of sharing the backbone.

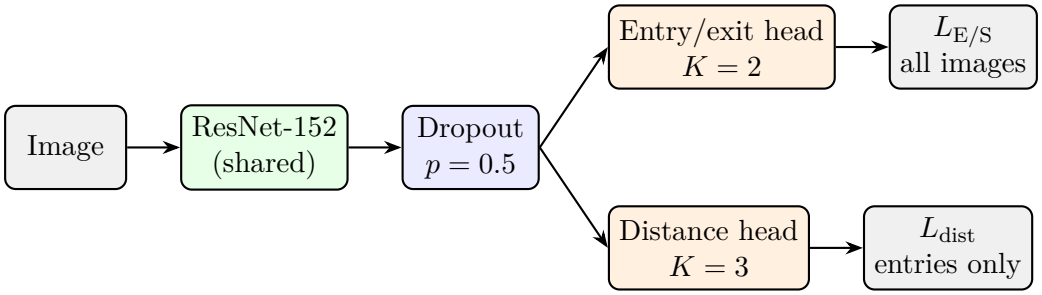


Figure 4.2: The multitask variant: one ResNet-152 and one Dropout shared by two heads of the same type (KAN or MLP-128). The loss is $L = L_{\mathrm{E/S}} + L_{\mathrm{dist}}$, with exit wounds masked out of the distance term.

Chapter 5

Methodology

This chapter describes the single evaluation protocol under which every model of this work was trained and tested. All models share the same images, the same five case-grouped folds, the same training recipe, the same metrics and the same statistical procedure; only the component under study (the head, the backbone, the split or the number of tasks) changes from one experiment to another. The protocol was written as one Python module (`protocolo_comum.py`, version 1.0.0, SHA-256 `b7cd9bac...`) and frozen before the first training run. The later experiments (shooting distance, the reproduction of the split of Lira et al., the wavelet-KAN heads and the multitask network) were added as plug-ins that register new heads, configurations or losses without editing the module, and every run records the hashes of the module, of the plug-in and of the fold file, so that it can be checked that all numbers come from the same code.

5.1 Dataset

The images come from the FDCPUnBGunshotDB [2], a collection of photographs of gunshot wounds taken at crime scenes and autopsies by the Federal District Civil Police (PCDF), Brazil. The database has no redistribution licence and its README requires citing Lira et al. [2]; it was cloned at a fixed commit (`f679389`) in every session [23], and no image or image cache left the training sessions.

Labels are read from the folder structure. The snapshot used contains 2,554 images from 486 cases: 1,883 entry wounds and 671 exit wounds. The 1,883 entries are further labelled with the medico-legal shooting distance (MLSD): 1,609 distant, 232 close-range and 42 contact shots; the 42 contact images come from only 16 cases. Lira et al. describe 2,551 images (1,883 entries and 668 exits) from 489 cases [2]. The difference in images comes from three exit images that are present in the folders but absent from the CSV distributed with the database; the CSV lists 2,551 images from 485 cases. The 489 cases

of the article are not reproduced by the distributed data. In addition, cases are identified here by year and case number (the first nine characters of the file name), whereas the code of Lira et al. uses the case number without the year, which merges 56 numbers that occur in more than one year (417 groups instead of 486) [21]. One exit image is labelled with a distance in the CSV; the folder label (exit) is used. The age distribution of the victims is shown in Figure 5.1.

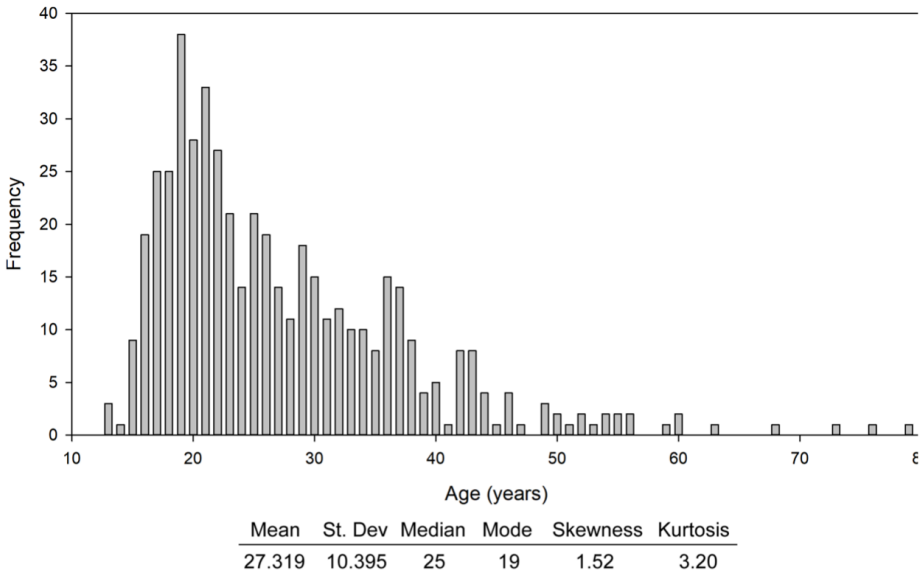
5.2 Case-Grouped Folds

Photographs of the same case (often several views of the same wound, or of wounds of the same victim) are strongly correlated. If images of one case fall in training and test, the test set no longer measures generalisation to new cases (Section 2.4). All experiments therefore use one set of five folds in which every case belongs to exactly one fold. The folds were generated once with `StratifiedGroupKFold` from scikit-learn [24] (five folds, shuffled, seed 42) over the image list sorted by path, with groups given by the case and stratification by a four-class label (exit, contact, close, distant), so that both tasks are balanced at the same time. A composition rule was fixed before the folds were generated: every test fold must contain at least two contact cases and at least one image of each class, and if seed 42 failed, the first seed ≥ 43 that passed would be used. Seed 42 passed. Table 5.1 shows the result; the share of exits per fold stays between 24.5% and 28.9% (26.3% overall).

Table 5.1: Composition of the five test folds (`StratifiedGroupKFold`, seed 42, groups = case, stratified by the four-class label). Contact: images (cases). No case appears in more than one fold.

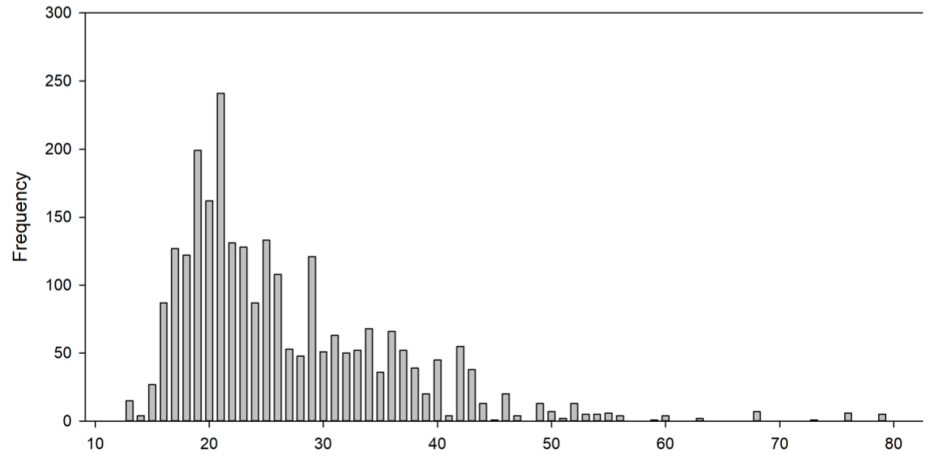
Fold	Images	Cases	Exits	Contact	Close	Distant
1	506	96	146	4 (2)	53	303
2	459	95	118	10 (4)	25	306
3	531	99	133	13 (5)	43	342
4	538	100	132	11 (3)	61	334
5	520	96	142	4 (2)	50	324
Total	2,554	486	671	42 (16)	232	1,609

The distance task uses the same folds restricted to the entries, so no case crosses folds in either task, and the two tasks can be compared and combined image by image.



Mean	St. Dev	Median	Mode	Skewness	Kurtosis
27.319	10.395	25	19	1.52	3.20

(a) Selected cases.



Mean	St. Dev	Median	Mode	Skewness	Kurtosis
26.680	9.557	24	21	1.63	4.11

(b) Selected images.

t time of death

Figure 5.1: Frequency of age at the time of death in the FDCPUnBGunshotDB dataset, reported by selected cases (a) and by selected images (b) (Source: [2]).

5.3 Training Recipe

Every model (heads, backbones, wavelet-KAN heads and multitask network) is trained with the recipe of the original TransferKAN notebook, which is now applied to all of them:

- **Model:** a torchvision [25] backbone with its default ImageNet weights [7], followed by flattening, Dropout ($p = 0.5$) and the head.
- **Phase 1 (epochs 1–15):** backbone frozen (batch-normalisation statistics still updated, as in the original code); only the new parameters are optimised with AdamW [26] (learning rate 10^{-3}, weight decay 10^{-2}) and cosine annealing [27] over 15 epochs.
- **Phase 2 (epochs 16–60):** the whole network is trained with AdamW (learning rate 10^{-5}, weight decay 10^{-3}) and cosine annealing over 45 epochs.
- **Batches and loss:** batches of 32 images, dropping the last incomplete batch; cross-entropy with class weights $n/(K\,n_c)$ computed on the training part of each fold (K classes, n_c images of class c).
- **No internal validation and no epoch selection:** the whole training part of the fold is used for training, and the model evaluated is the one at epoch 60. The test fold is evaluated once.
- **Augmentation** (training only, Albumentations [28] 2.0.8, with the values that actually ran in the original notebook): resize to 224×224; horizontal and vertical flips ($p = 0.5$ each); shift-scale-rotate (shift 0.05, scale 0.1, rotation up to 30°, $p = 0.5$); CLAHE (clip 2.0, 8×8 tiles, $p = 0.3$); elastic transform ($\alpha = 1$, $\sigma = 50$, $p = 0.3$); Gaussian noise (standard deviation between 0.2 and 0.44 of the intensity range, $p = 0.2$); colour jitter (brightness, contrast and saturation 0.2, hue 0.05, $p = 0.5$); ImageNet normalisation. Test images are only resized and normalised.
- **Seeds:** the folds are fixed; the training seed (42, 43 or 44) controls the head initialisation, the dropout masks, the batch order and the augmentation.

The original notebook was written for an older Albumentations interface; two of its arguments (`alpha_affine` and `var_limit`) are silently discarded by version 2, so the elastic transform and the Gaussian noise ran with the values listed above rather than with the values written in the notebook. The protocol fixes those effective values explicitly, so the augmentation is identical to the one that produced the original results and is the same for every model.

Numerical precision and hardware. All comparable runs used an NVIDIA L4 GPU on Google Colab. The backbone runs in automatic mixed precision (fp16) [29] and the heads always in fp32. Mixed precision was adopted only after a check defined before any run: the TransferKAN was trained on fold 1 with seed 42 in fp32 and in mixed precision, and mixed precision would be accepted if accuracy changed by at most 2 percentage points and AUC by at most 0.015. Accuracy was 84.98% in fp32 and 84.39% in mixed precision, and AUC 0.9078 and 0.9076, while the time per fold fell from 14.9 to 8.1 minutes; mixed precision was therefore used for every model.

5.4 Heads Compared on the ResNet-152

The head ablation keeps the ResNet-152 backbone (58,143,808 parameters), the folds and the recipe fixed and changes only the layer that maps the 2,048 features to the classes. The five heads, with their parameter counts, are listed in Table 4.1 (Chapter 4): (a) a linear layer; (b) an MLP with 128 hidden units; (c) an MLP whose hidden width (1,919) gives it practically the same number of parameters as the KAN (3.94 million); (d) the KAN head of TransferKAN, $[2048, 128, K]$ with grid size 10 and cubic splines, implemented with `efficient-kan` [10] (15 parameters per edge: 13 B-spline coefficients, one base weight and one spline scale); and (e) a small KAN, $[2048, 8, 2]$, with roughly the budget of (b). Heads (a)–(d) were trained with seeds 42, 43 and 44 in both tasks; head (e) only with seed 42 and only for entry/exit.

5.5 Backbones Under the Same Protocol

To place TransferKAN among the architectures evaluated by Lira et al., twelve of the 59 architectures of their Table 4 were retrained with the same recipe. They cover convolutional networks from 0.3 to 134 million parameters and two transformer-based architectures, and include their best model, the ResNet-152 [6] with a linear head, which is head (a). The other eleven are AlexNet [30], MobileNetV2 [31], MNASNet 1.3 [32], ShuffleNetV2 $\times$0.5 [33], SqueezeNet 1.0 [34], DenseNet-169 [35], MaxViT-T [9], Swin-B [8], EfficientNetV2-L [36], VGG-16 [37] and RegNetY-16GF [38]. The catalogue follows the code of Lira et al. [21]: the same torchvision constructors and the same point where the last layer is replaced. Two deviations are declared: every backbone starts from the default ImageNet weights (in the public code of Lira et al. only the ResNets are pre-trained, and their first convolution is replaced by a randomly initialised one), and the first convolution is kept. The new last layer is Dropout(0.5) followed by a linear layer (a 1×1 convolution in the SqueezeNet). EfficientNetV2-L, pre-trained at 480 px, runs at 224 px like the

others. Swin-B and MaxViT-T were trained with seeds 42, 43 and 44 for entry/exit and with seeds 42 and 43 for the shooting distance; the other backbones with seed 42 only.

5.6 Shooting Distance

The distance task (contact, close range, distant) uses only entry wounds, since exits carry no distance information. For test fold k, the network is trained on the entries of the other four folds and evaluated on the entries of fold k. The heads have $K = 3$ outputs, and everything else (recipe, seeds, grid size 10 of the KAN) is as in the entry/exit task. The earlier distance model of this work, trained in a single phase with a grid of 5 and with the epoch chosen on the test set, was discarded and is not used anywhere in this dissertation.

5.7 Reproducing the Split of Lira et al.

To measure how much the split alone changes the results, the hold-out of Lira et al. was reproduced from their public code (`iterative_split` in `leguwoi`, commit `83d82f1`) [21]. That function splits images 80/20 with iterative stratification [14] (`IterativeStratification` of scikit-multilearn 0.2.0, order 2) on two columns, the case number and the label. Stratifying on the case number spreads the images of each case over training and test, instead of keeping them together. The function fixes no seed, so three partitions were generated with seeds 42, 43 and 44, on the 2,551 images of the distributed CSV (entry/exit: 2,074 training and 477 test images; distance: 1,520 and 363 entries). The ResNet-152 with heads (a) and (d) was trained on each partition with the recipe above (seed 42) and compared with the same models on the case-grouped folds (seed 42). This reproduces the method of the public code, not necessarily the exact partition used for the published tables, which cannot be recovered.

5.8 Wavelet-KAN Heads

To test whether a different family of KAN changes the conclusion of the head ablation, wavelet-KAN (Wav-KAN) heads [11] were added. Each layer follows the `WavKANLayer` of `torch-conv-kan` [22]: every edge has a learnable scale, a learnable translation, a wavelet weight and a weight for a SiLU base branch, and the layer output goes through batch normalisation. The implementation used here was checked against `torch-conv-kan` (commit `b412a3b`) and gave identical outputs and gradients on CPU and GPU. The heads have two layers, $[2048, h, K]$, and $8202h + 4$ parameters for $K = 2$. Five configurations were trained (Table 4.1): W1, Mexican hat with $h = 128$ (the topology of head (d), with

3.7 times fewer parameters); W2, Mexican hat with $h = 32$ (the budget of the MLP (b)); W3 and W4, derivative-of-Gaussian and Morlet wavelets with $h = 128$ (sensitivity to the wavelet); and W5, Mexican hat with $h = 480$ (the budget of (d)). The Mexican hat was fixed in advance for the matched budgets. W1 and W2 were trained with seeds 42–44 in both tasks; W3–W5 only with seed 42 and only for entry/exit. Besides the time per fold, the head alone was benchmarked on random 2048-dimensional inputs (forward and backward pass for a batch of 32 and inference per image; 20 warm-up and 200 timed runs with CUDA events).

5.9 One Network for Both Tasks

The two tasks are currently solved by two separate networks. The multitask experiment trains one ResNet-152 with a shared backbone and Dropout and two heads, one for entry/exit ($K = 2$) and one for distance ($K = 3$) [16, 17]. Both heads are of the same type, either the KAN (d) or the MLP-128 (b). The loss is $L = L_{\mathrm{E/S}} + L_{\mathrm{dist}}$: each term is a class-weighted cross-entropy, and exits are masked out of the distance term (their distance label is undefined), so the entry/exit head learns from all 2,554 images and the distance head from the 1,883 entries. When a batch contains no entry, the distance term is set to zero. The single-task references are the networks of the head ablation, (d) and (b), for each task, trained with the same seeds (42–44); the comparison is therefore paired on the same test images.

Two further measures are reported. A four-class pipeline (exit, contact, close, distant) combines the two outputs: with the single-task pair, the entry/exit network decides and, for predicted entries, the distance network decides; with the multitask network, its two heads do the same. This measure is secondary and descriptive. The cost is reported as parameters, training time per fold and the time to obtain both outputs (one forward pass of the multitask network against two forward passes of the pair), measured on the L4 in mixed precision with batches of 1 and 32 (20 warm-up and 200 timed runs). Inference time does not depend on the weights and was measured on freshly built models; no trained weights were stored.

5.10 Metrics

For each fold and each seed, the following are computed on the test images: accuracy, balanced accuracy, macro precision, recall and F1, and the area under the ROC curve (AUC) [15] computed from the predicted probabilities. In the binary task the AUC uses the probability of the exit class; in the distance task it is the macro average of the three

one-vs-rest AUCs, computed on entries only. Results are reported as mean ± standard deviation across the five folds, where each fold is first averaged over the seeds; the pooled out-of-fold AUC (AUC_{OOF}), computed after averaging each image's probabilities over the seeds, is also reported. In the distance task accuracy is misleading: always predicting "distant" already gives 0.854. The main metrics are therefore AUC and macro F1.

Lira et al. computed their AUC from predicted classes rather than probabilities [21]. For a single threshold the ROC curve has one point, and the macro AUC reduces to the average of recall and specificity; this quantity, called AUC_c here, is computed only to compare with their tables (Section 2.5).

5.11 Statistics and Pre-Registered Decision Rules

Two models are always compared on the same test images. The difference Δ (first minus second) is computed on the pooled out-of-fold predictions, and its 95% confidence interval comes from a paired bootstrap over cases [18]: cases are resampled with replacement 2,000 times, which respects the dependence between images of the same case. The difference per fold and the number of folds in which each model wins are also reported.

The decision rules were written before the corresponding runs. The rules of the head ablation, the backbones and the distance task were fixed in the experiment plans and in the analysis notebooks committed to the repository on 14 October 2026 (commits `9961146` and `367a7a6`, which remain in its history), before the first training run:

- **Primary comparison:** the KAN head (d) against the better of the two MLPs, (b) or (c), in AUC.
- **Outcome A (the KAN improves):** ΔAUC > 0 in at least four of the five folds *and* a confidence interval above zero.
- **Outcome C (the KAN is worse):** the KAN wins at most one fold *and* the interval is below zero.
- **Outcome B (equivalent):** otherwise.
- **Claims forbidden without the corresponding result:** "only changing the head improves", "new state of the art", "outperforms the transformer baselines" and "parameter efficiency".

The wavelet-KAN and multitask experiments were planned in the same commit of plans (`9961146`), with a non-inferiority margin of -0.02: a model is *non-inferior* when the lower bound of the interval of ΔAUC (and, for the wavelet-KAN heads, also of ΔF1) is above -0.02. Superior and inferior follow the rule of outcomes A and C. For the

wavelet-KAN heads, the phrase "more parameter-efficient than an MLP" was allowed only if W2 were superior to the MLP (b) of the same size, and "faster" only if both the head benchmark and the time per fold improved beyond run-to-run noise. On 19 October 2026, after the first experiment and before any wavelet-KAN or multitask run, three adjustments were recorded (commit `3266a99`): the main configurations would use seeds 42–44, as the references; W1 and W2 would also be run on the distance task; and the multitask comparison would reuse the single-task networks of the head ablation. The margin was kept.

5.12 Threats to Validity

No validation split. Evaluating the last epoch avoids selecting the epoch on the test set, which inflated the earlier results of this work, but it is not tuned either: a model stopped by an inner validation split could be slightly better. The rule is the same for every model, so it does not favour any of them.

Augmentation as it ran. The Gaussian noise of the original recipe is strong (standard deviation up to 0.44 of the intensity range) and may erase fine traces such as stippling, which matter for the close-range class. It applies equally to all models; it is a limitation of the recipe, not a bias of the comparisons.

Seeds. The head ablation, Swin-B and MaxViT-T (entry/exit), W1, W2 and the multitask networks use three seeds. The other backbones, head (e) and W3–W5 use one seed, and Swin-B and MaxViT-T use two seeds in the distance task; these results are reported with that caveat.

The split of Lira et al. The leakage experiment reproduces the method of the public code with three seeds; the exact partition behind the published tables cannot be recovered, and the code may differ from the one used for the article.

Statistical power and the contact class. Five folds give little power to detect small differences between heads. The contact class has 42 images from 16 cases (4 to 13 test images per fold), so nothing specific is claimed about it.

A single database. All images come from one police force and one region; no external validation was possible.

Chapter 6

Results

This chapter reports the experiments of Chapter 5 in the order of the research questions: first the effect of the split used in 2024 (Section 6.1), then the head ablation and the backbones in the two tasks (Sections 6.2 and 6.3), the comparison with the published numbers of Lira et al. (Section 6.4) and with the transformers (Section 6.5), and finally the wavelet-KAN heads (Section 6.6) and the multitask network (Section 6.7). Unless stated otherwise, numbers are means ± standard deviations across the five case-grouped folds, AUC is computed from probabilities, and differences come with 95% confidence intervals from the paired case-level bootstrap.

6.1 The 2024 Hold-Out Leaks Cases

Reproducing the hold-out of the public code of Lira et al. in three partitions, 98.7% of the entry/exit test images and 98.4% of the distance test images belong to a case that also has images in the training set. Table 6.1 and Figure 6.1 compare, with the same network, the same recipe and the same seed, the results on those partitions and on the case-grouped folds.

On entry/exit, the AUC rises from 0.906 on the grouped folds to 0.940 on the leaky partitions for both heads (+0.034). On the shooting distance the effect is about five times larger: from 0.779 to 0.937 with the linear head (+0.158) and from 0.788 to 0.942 with the KAN head (+0.155); balanced accuracy goes from about 0.51 to about 0.78. The leaky partitions also look far more stable (standard deviation of 0.003–0.004 in AUC, against 0.011–0.013 in entry/exit and 0.06–0.09 in distance across grouped folds). In other words, a split that lets the network see other photographs of the same case makes the distance task look almost as easy as entry/exit, while case-grouped folds show it is much harder. The grouped numbers in this table use seed 42 only, so they differ slightly

Table 6.1: Effect of the data split with everything else fixed (ResNet-152, same recipe, seed 42). *Lira et al.*: the image-level hold-out of the public code of Lira et al. (iterative stratification on case number and label) reproduced with three partitions; mean ± s.d. over the partitions. *Grouped*: the five case-grouped folds of this work; mean ± s.d. over folds. Leak: share of test images whose case also has images in training. Δ = Lira − grouped.

Task	Head	Split	Leak	Acc	$F1_M$	Bal. acc	AUC
Entry/exit	Linear (a)	Lira et al.	98.7%	0.876 ± 0.005	0.838 ± 0.007	0.852 ± 0.010	0.940 ± 0.003
		Grouped	0%	0.848 ± 0.010	0.809 ± 0.013	0.820 ± 0.011	0.906 ± 0.013
		Δ		+0.028	+0.029	+0.033	+0.034
Entry/exit	KAN (d) = TransferKAN	Lira et al.	98.7%	0.881 ± 0.006	0.844 ± 0.006	0.859 ± 0.003	0.940 ± 0.003
		Grouped	0%	0.846 ± 0.008	0.807 ± 0.011	0.818 ± 0.013	0.906 ± 0.011
		Δ		+0.035	+0.037	+0.040	+0.034
MLSD	Linear (a)	Lira et al.	98.4%	0.857 ± 0.006	0.697 ± 0.014	0.783 ± 0.008	0.937 ± 0.004
		Grouped	0%	0.817 ± 0.014	0.486 ± 0.059	0.521 ± 0.079	0.779 ± 0.088
		Δ		+0.040	+0.211	+0.262	+0.158
MLSD	KAN (d) = TransferKAN	Lira et al.	98.4%	0.863 ± 0.002	0.691 ± 0.003	0.788 ± 0.006	0.942 ± 0.003
		Grouped	0%	0.810 ± 0.018	0.466 ± 0.035	0.500 ± 0.062	0.788 ± 0.063
		Δ		+0.053	+0.225	+0.288	+0.155

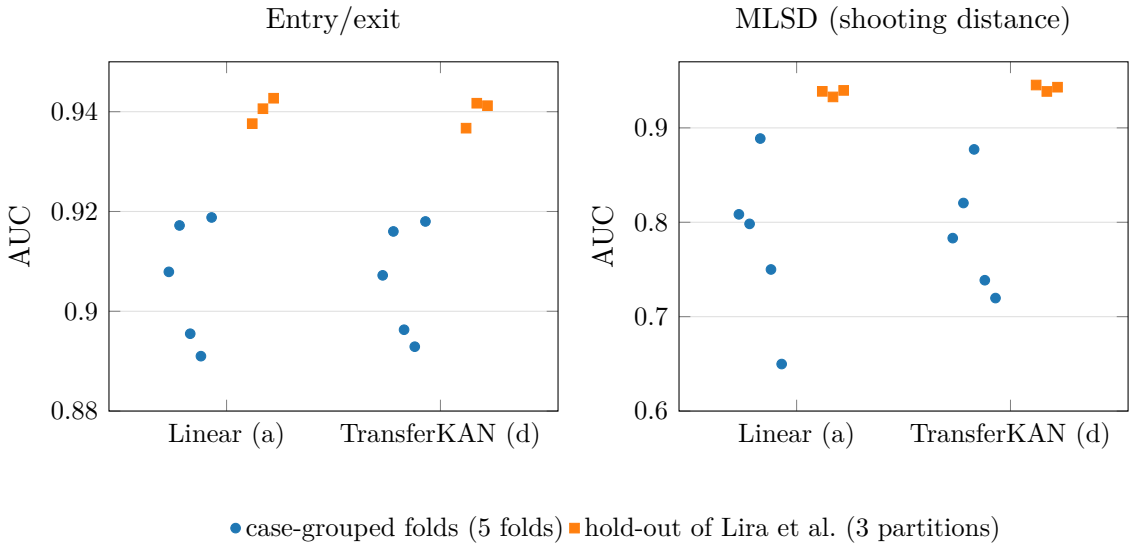


Figure 6.1: AUC of the ResNet-152 with the linear head (a) and with the KAN head (d), seed 42, on each case-grouped fold (circles) and on each reproduced partition of Lira et al. (squares). Same network, recipe and seed; only the split changes.

from the three-seed numbers of the following sections (0.906 instead of 0.904 for the linear head).

The k-fold analysis that Lira et al. report for their ResNet-152 points the same way: with folds grouped by case, the median accuracy of their model in entry/exit is about 0.70 (read from their boxplot, Figure 6.2), well below the 0.869 of their hold-out table.

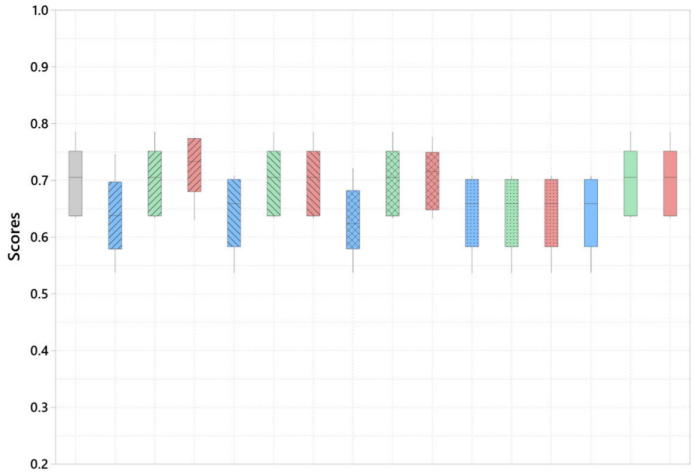
6.2 Entry/Exit: Heads and Backbones

Table 6.2 lists all entry/exit results under the single protocol.

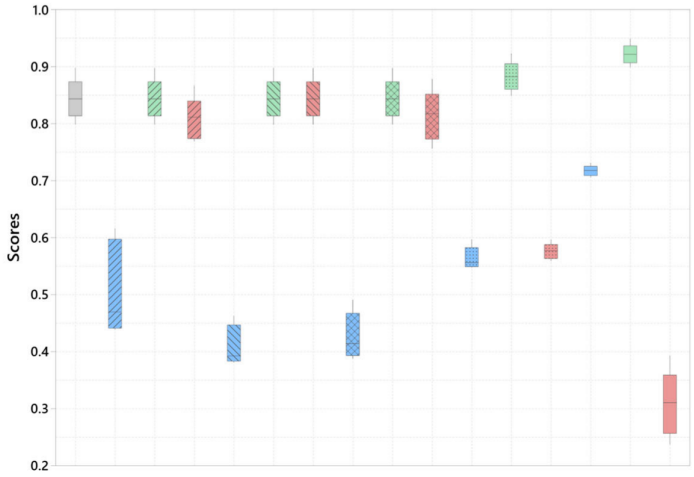
Table 6.2: Entry/exit classification under the single protocol (case-grouped stratified 5-fold, same two-phase recipe, last epoch). Mean ± s.d. across folds; each fold is the mean over the training seeds (column S). F1 is the macro average (macro precision and recall are in Appendix A); AUC is computed from predicted probabilities. Min: training minutes per fold on an NVIDIA L4.

Model	Params (M)	S	Acc	F1	AUC	Min
Head ablation (ResNet-152 backbone)						
ResNet152 + Linear (a)	58.15	3	0.846 ± 0.008	0.805 ± 0.012	0.904 ± 0.013	8.2
ResNet152 + MLP-128 (b)	58.41	3	0.846 ± 0.011	0.806 ± 0.015	0.905 ± 0.014	8.3
ResNet152 + MLP-iso (c)	62.08	3	0.846 ± 0.009	0.807 ± 0.013	0.906 ± 0.014	8.4
ResNet152 + KAN (d) = TransferKAN	62.08	3	0.846 ± 0.010	0.806 ± 0.013	0.905 ± 0.013	8.5
ResNet152 + KAN-8 (e)	58.39	1	0.846 ± 0.012	0.808 ± 0.014	0.903 ± 0.014	8.4
Backbones (linear head)						
ResNet152	58.15	3	0.846 ± 0.008	0.805 ± 0.012	0.904 ± 0.013	8.2
AlexNet	57.01	1	0.803 ± 0.036	0.761 ± 0.047	0.859 ± 0.040	3.2
MobileNetV2	2.23	1	0.812 ± 0.024	0.776 ± 0.029	0.877 ± 0.025	3.9
MNASNet1.3	5.00	1	0.837 ± 0.012	0.753 ± 0.022	0.872 ± 0.021	3.9
ShuffleNetV2 x0.5	0.34	1	0.746 ± 0.018	0.713 ± 0.018	0.839 ± 0.021	3.8
SqueezeNet1.0	0.74	1	0.798 ± 0.023	0.760 ± 0.030	0.866 ± 0.023	3.2
DenseNet169	12.49	1	0.858 ± 0.005	0.821 ± 0.008	0.904 ± 0.018	8.4
MaxViT-T	30.41	3	0.864 ± 0.015	0.831 ± 0.021	0.917 ± 0.009	12.4
Swin-B	86.75	3	0.884 ± 0.011	0.850 ± 0.015	0.929 ± 0.011	15.4
EfficientNetV2-L	117.24	1	0.868 ± 0.010	0.831 ± 0.016	0.916 ± 0.007	15.2
VGG16	134.27	1	0.841 ± 0.013	0.794 ± 0.025	0.891 ± 0.025	8.6
RegNetY-16GF	80.57	1	0.852 ± 0.015	0.815 ± 0.020	0.906 ± 0.011	12.1
ResNet152 + KAN (d) = TransferKAN	62.08	3	0.846 ± 0.010	0.806 ± 0.013	0.905 ± 0.013	8.5

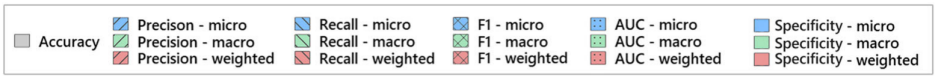
Heads. With the ResNet-152 fixed, the five heads are indistinguishable: AUC from 0.903 (KAN-8) to 0.906 (matched MLP), and accuracy 0.846 for all of them. The pre-registered rule gives outcome B. The best MLP is the matched one (c), and the KAN head (d) differs from it by $\Delta\text{AUC} = -0.001$ (95% CI $[-0.002, +0.0003]$), winning 2 of the 5 folds. Against the MLP-128 (b) the difference is -0.0001 ($[-0.001, +0.001]$, 3 of 5 folds), and against the linear head (a) it is $+0.001$ ($[-0.001, +0.003]$, 3 of 5 folds); all intervals include zero. The ROC curves of the four heads practically coincide (Figure 6.3). Under



(a) Wound type



(b) Medical-legal shooting distance



ation performance for wound type (a) and shooting distance (b) using ResNet 152 over the k-folds

Figure 6.2: Stability analysis of the ResNet-152 of Lira et al. over grouped k-folds, for wound type (a) and shooting distance (b); the leftmost (grey) box of each panel is accuracy (Source: [2]).

the same protocol, then, the KAN head performs on par with linear and MLP heads; it is not the source of the results of TransferKAN.

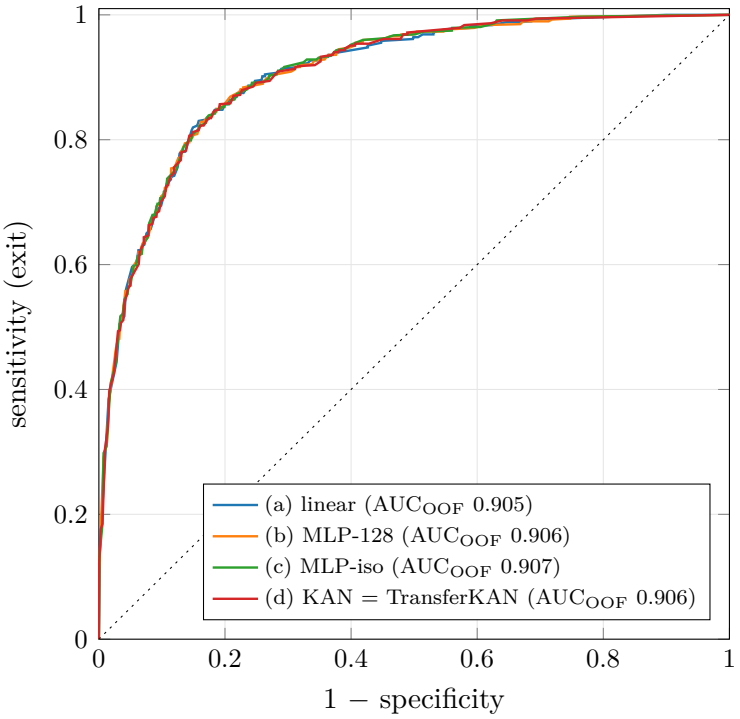


Figure 6.3: Out-of-fold ROC curves of the four heads on the ResNet-152 (entry/exit; probabilities averaged over seeds 42–44; exit is the positive class).

A diagnostic recorded during evaluation suggests why the spline brings little. `efficient-kan` defines the B-splines only on a grid over $[-1, 1]$ (slightly extended at the borders); outside it, each edge reduces to its SiLU base branch, which works like an MLP. On the test images, 97–98% of the inputs of the first KAN layer fall inside the grid, but only 46–64% of the inputs of the second layer do (27–39% in the small KAN (e)), so a large part of the second layer works without its splines.

Backbones. The pre-trained backbones spread from 0.839 (ShuffleNetV2 ×0.5) to 0.929 (Swin-B). Swin-B is the best model of the study (AUC 0.929±0.011, accuracy 0.884), followed by MaxViT-T (0.917) and EfficientNetV2-L (0.916). TransferKAN (0.905) sits with RegNetY-16GF (0.906), DenseNet-169 (0.904) and the ResNet-152 with a linear head (0.904), in the middle of the table. The remaining networks (VGG-16, MobileNetV2, MNASNet, SqueezeNet, AlexNet and ShuffleNetV2) stay at or below 0.891. Except for the ResNet-152 heads, Swin-B and MaxViT-T, these rows use a single seed.

6.3 Shooting Distance: Heads and Backbones

Table 6.3 gives the same comparison for the distance task.

Table 6.3: Medico-legal shooting distance (contact, close range, distant; entry wounds only) under the same protocol as Table 6.2. Mean ± s.d. across folds; S = training seeds (Swin-B and MaxViT-T: seeds 42 and 43 only). AUC = one-vs-rest macro AUC from probabilities. Rec.: per-class recall (mean across folds); contact has 4–13 test images per fold and is descriptive only.

Model	Params (M)	S	Acc	Bal. acc	$F1_M$	AUC	Rec. cont.	Rec. close	Min
Head ablation (ResNet-152 backbone)									
ResNet152 + Linear (a)	58.15	3	0.815 ± 0.014	0.524 ± 0.077	0.488 ± 0.056	0.785 ± 0.087	0.17	0.53	6.3
ResNet152 + MLP-128 (b)	58.41	3	0.813 ± 0.013	0.520 ± 0.072	0.482 ± 0.050	0.802 ± 0.062	0.16	0.53	6.3
ResNet152 + MLP-iso (c)	62.08	3	0.821 ± 0.017	0.510 ± 0.064	0.480 ± 0.050	0.807 ± 0.055	0.13	0.51	6.4
ResNet152 + KAN (d) = TransferKAN	62.08	3	0.813 ± 0.017	0.515 ± 0.069	0.477 ± 0.045	0.798 ± 0.058	0.15	0.52	6.5
Backbones (linear head)									
ResNet152	58.15	3	0.815 ± 0.014	0.524 ± 0.077	0.488 ± 0.056	0.785 ± 0.087	0.17	0.53	6.3
AlexNet	57.02	1	0.780 ± 0.033	0.561 ± 0.105	0.498 ± 0.084	0.806 ± 0.079	0.26	0.59	2.5
MobileNetV2	2.23	1	0.778 ± 0.025	0.513 ± 0.074	0.464 ± 0.042	0.806 ± 0.056	0.20	0.50	3.1
MNASNet1.3	5.01	1	0.451 ± 0.078	0.455 ± 0.051	0.294 ± 0.047	0.710 ± 0.095	0.10	0.86	3.1
ShuffleNetV2 x0.5	0.34	1	0.854 ± 0.016	0.474 ± 0.079	0.474 ± 0.088	0.776 ± 0.046	0.10	0.38	3.1
SqueezeNet1.0	0.74	1	0.788 ± 0.023	0.525 ± 0.090	0.490 ± 0.074	0.805 ± 0.051	0.20	0.53	2.6
DenseNet169	12.49	1	0.820 ± 0.034	0.556 ± 0.136	0.519 ± 0.120	0.821 ± 0.070	0.24	0.55	6.6
MaxViT-T	30.41	2	0.810 ± 0.021	0.555 ± 0.086	0.511 ± 0.070	0.828 ± 0.047	0.23	0.58	9.4
Swin-B	86.75	2	0.825 ± 0.021	0.540 ± 0.087	0.505 ± 0.091	0.838 ± 0.054	0.18	0.55	11.4
EfficientNetV2-L	117.24	1	0.830 ± 0.026	0.529 ± 0.084	0.495 ± 0.076	0.744 ± 0.133	0.15	0.55	11.6
VGG16	134.27	1	0.844 ± 0.026	0.510 ± 0.067	0.481 ± 0.050	0.776 ± 0.079	0.10	0.51	6.6
RegNetY-16GF	80.57	1	0.826 ± 0.024	0.531 ± 0.080	0.496 ± 0.071	0.773 ± 0.088	0.13	0.58	9.1
ResNet152 + KAN (d) = TransferKAN	62.08	3	0.813 ± 0.017	0.515 ± 0.069	0.477 ± 0.045	0.798 ± 0.058	0.15	0.52	6.5

Heads. The AUCs are 0.785 ± 0.087 (linear), 0.802 ± 0.062 (MLP-128), 0.807 ± 0.055 (matched MLP) and 0.798 ± 0.058 (KAN). The pre-registered rule gives outcome C: against the best MLP, the matched one, the KAN head has $\Delta\text{AUC} = -0.011$ ($[-0.021, -0.003]$) and wins only 1 of the 5 folds. The KAN is significantly below *only* that MLP: against the MLP-128 the difference is -0.005 ($[-0.012, +0.003]$), and against the linear head $+0.008$ ($[-0.005, +0.023]$), both intervals including zero. The per-class ROC curves (Figure 6.4) overlap for close and distant shots and diverge only for contact, which has 42 images.

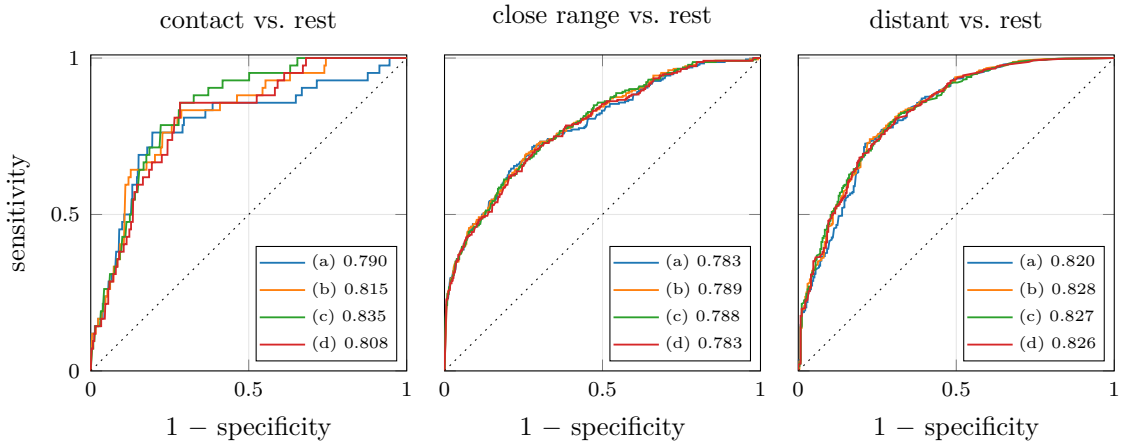


Figure 6.4: Out-of-fold one-vs-rest ROC curves of the four heads on the ResNet-152 for the shooting distance (entries only; probabilities averaged over seeds 42–44). The legend gives each head's one-vs-rest AUC.

All heads have balanced accuracy around 0.51–0.52 and macro F1 around 0.48–0.49. Recall is about 0.88 for distant and 0.51–0.53 for close-range shots, and only 0.13–0.17 for contact. The contact figure is descriptive: there are 4 to 13 contact images per test fold. When the case of a contact wound is not in the training set, the networks almost never recognise it. This is the main limitation of the present models.

Backbones. Swin-B is again the best (0.838 ± 0.054), followed by MaxViT-T (0.828) and DenseNet-169 (0.821); both transformers have only seeds 42 and 43 in this task. Older or smaller networks such as AlexNet, MobileNetV2 and SqueezeNet reach 0.805–0.806, above the ResNet-152, while EfficientNetV2-L (0.744) varies strongly across folds (± 0.133). The MNASNet collapsed in this task (accuracy 0.451, predicting most entries as close range), which is an optimisation instability with this recipe rather than a property of the architecture. With fold-to-fold standard deviations of 0.05–0.13, most differences between backbones in this task are within noise.

6.4 Comparison with the Published Numbers of Lira et al.

Tables 6.4 and 6.5 place the numbers of Tables 4 and 5 of Lira et al. next to the same backbones under the single protocol. The two sides differ in three ways at once (the split, the initialisation and the definition of the AUC), so the tables compare evaluation packages, not models; only the AUC from classes, AUC_c, is computed in the same way on both sides.

In entry/exit, the ResNet-152 of Lira et al. reports accuracy 0.869, macro F1 0.824 and AUC_c 0.821. The same backbone with a linear head under the single protocol obtains 0.846, 0.805 and 0.814 (AUC from probabilities 0.904). TransferKAN, with 0.846, 0.806 and 0.816, lies above eleven of the twelve published rows but *below* their ResNet-152 in all three metrics. For the other eleven architectures the single protocol gives higher numbers than the published ones in all three metrics, usually much higher (Swin-B: accuracy from 0.728 to 0.884; AUC_c from 0.591 to 0.850), although its split is the stricter one. This is consistent with the fact that in the public code only the ResNets start from ImageNet weights, but this work did not train from scratch, so the effect of pre-training is not isolated here.

In the distance task, the published accuracies (0.855–0.925 for all rows but SqueezeNet) are higher than any accuracy under the single protocol, and TransferKAN (0.813) is below eleven of the twelve published rows. In AUC_c, TransferKAN (0.664) is above eleven of the twelve rows and below their ResNet-152 (0.733). Several published models have AUC_c

Table 6.4: Entry/exit: numbers published by Lira et al. [2] (Table 4) next to the same backbones under the single protocol of this work. The two sides differ in three ways at once: the split (image-level hold-out in their public code × case-grouped 5-fold), the initialisation (mostly trained from scratch in their public code × ImageNet weights) and the AUC (from predicted classes, AUC_c × from probabilities, AUC_p). Only AUC_c is computed in the same way on both sides; the table compares evaluation packages, not models. M = macro; ours: mean ± s.d. across folds.

	Lira et al. (published)			Single protocol (this work)			
Model	Acc	$F1_M$	$AUC_{c,M}$	Acc	$F1_M$	$AUC_{c,M}$	$AUC_{p,M}$
ResNet152	0.869	0.824	0.821	0.846 ± 0.008	0.805 ± 0.012	0.814 ± 0.012	0.904 ± 0.013
AlexNet	0.765	0.703	0.716	0.803 ± 0.036	0.761 ± 0.047	0.782 ± 0.045	0.859 ± 0.040
MobileNetV2	0.719	0.598	0.593	0.812 ± 0.024	0.776 ± 0.029	0.802 ± 0.027	0.877 ± 0.025
MNASNet1.3	0.746	0.657	0.656	0.837 ± 0.012	0.753 ± 0.022	0.725 ± 0.023	0.872 ± 0.021
ShuffleNetV2 x0.5	0.730	0.619	0.614	0.746 ± 0.018	0.713 ± 0.018	0.755 ± 0.017	0.839 ± 0.021
SqueezeNet1.0	0.750	0.429	0.500	0.798 ± 0.023	0.760 ± 0.030	0.787 ± 0.032	0.866 ± 0.023
DenseNet169	0.777	0.688	0.679	0.858 ± 0.005	0.821 ± 0.008	0.830 ± 0.012	0.904 ± 0.018
MaxViT-T	0.780	0.668	0.653	0.864 ± 0.015	0.831 ± 0.021	0.846 ± 0.020	0.917 ± 0.009
Swin-B	0.728	0.597	0.591	0.884 ± 0.011	0.850 ± 0.015	0.850 ± 0.016	0.929 ± 0.011
EfficientNetV2-L	0.782	0.654	0.638	0.868 ± 0.010	0.831 ± 0.016	0.836 ± 0.011	0.916 ± 0.007
VGG16	0.782	0.694	0.685	0.841 ± 0.013	0.794 ± 0.025	0.796 ± 0.034	0.891 ± 0.025
RegNetY-16GF	0.711	0.549	0.549	0.852 ± 0.015	0.815 ± 0.020	0.824 ± 0.021	0.906 ± 0.011
TransferKAN (ResNet152 + KAN)	—	—	—	0.846 ± 0.010	0.806 ± 0.013	0.816 ± 0.015	0.905 ± 0.013

Table 6.5: MLSD (entries only): numbers published by Lira et al. [2] (Table 5) next to the same backbones under the single protocol of this work. The two sides differ in three ways at once: the split (image-level hold-out in their public code × case-grouped 5-fold), the initialisation (mostly trained from scratch in their public code × ImageNet weights) and the AUC (from predicted classes, AUC_c × from probabilities, AUC_p). Only AUC_c is computed in the same way on both sides; the table compares evaluation packages, not models. M = macro; ours: mean ± s.d. across folds. In this task, predicting always "distant" already gives an accuracy of 0.854.

	Lira et al. (published)			Single protocol (this work)			
Model	Acc	$F1_M$	$AUC_{c,M}$	Acc	$F1_M$	$AUC_{c,M}$	$AUC_{p,M}$
ResNet152	0.925	0.701	0.733	0.815 ± 0.014	0.488 ± 0.056	0.670 ± 0.047	0.785 ± 0.087
AlexNet	0.889	0.395	0.541	0.780 ± 0.033	0.498 ± 0.084	0.694 ± 0.064	0.806 ± 0.079
MobileNetV2	0.886	0.361	0.531	0.778 ± 0.025	0.464 ± 0.042	0.660 ± 0.043	0.806 ± 0.056
MNASNet1.3	0.869	0.378	0.533	0.451 ± 0.078	0.294 ± 0.047	0.610 ± 0.037	0.710 ± 0.095
ShuffleNetV2 x0.5	0.855	0.346	0.512	0.854 ± 0.016	0.474 ± 0.088	0.627 ± 0.044	0.776 ± 0.046
SqueezeNet1.0	0.095	0.058	0.500	0.788 ± 0.023	0.490 ± 0.074	0.666 ± 0.056	0.805 ± 0.051
DenseNet169	0.883	0.398	0.548	0.820 ± 0.034	0.519 ± 0.120	0.691 ± 0.092	0.821 ± 0.070
MaxViT-T	0.880	0.549	0.607	0.810 ± 0.021	0.511 ± 0.070	0.691 ± 0.053	0.828 ± 0.047
Swin-B	0.861	0.339	0.517	0.825 ± 0.021	0.505 ± 0.091	0.684 ± 0.053	0.838 ± 0.054
EfficientNetV2-L	0.880	0.488	0.588	0.830 ± 0.026	0.495 ± 0.076	0.673 ± 0.057	0.744 ± 0.133
VGG16	0.864	0.381	0.546	0.844 ± 0.026	0.481 ± 0.050	0.656 ± 0.041	0.776 ± 0.079
RegNetY-16GF	0.875	0.357	0.523	0.826 ± 0.024	0.496 ± 0.071	0.678 ± 0.048	0.773 ± 0.088
TransferKAN (ResNet152 + KAN)	—	—	—	0.813 ± 0.017	0.477 ± 0.045	0.664 ± 0.041	0.798 ± 0.058

close to 0.5 (SqueezeNet 0.500, ShuffleNetV2 0.512), that is, they hardly separate the classes even with high accuracy, which in this task is dominated by the distant class.

These comparisons support a narrower statement than the one made in the earlier version of this work. Under the single protocol, TransferKAN is equivalent to the same ResNet-152 with a linear head and below the best pre-trained backbones (Section 6.5). The differences from the numbers of Lira et al. come from the split, the pre-training and the recipe, not from the KAN head. Only the effect of the split was isolated with the same recipe (Section 6.1).

6.5 TransferKAN and the Transformers

Because the two transformers were the best backbones, TransferKAN was compared with them directly on the same images (Table 6.6).

Table 6.6: TransferKAN (ResNet-152 + spline KAN, seeds 42–44) against the two transformer backbones with a linear head, under the single protocol. ΔAUC on the pooled out-of-fold predictions (probabilities averaged over the seeds); 95% CI from a paired case-level bootstrap (2,000 resamples); wins = folds in which TransferKAN has the higher AUC.

Task	Contrast	ΔAUC	95% CI	Wins	Seeds of the transformer
Entry/exit	TransferKAN − Swin-B	−0.023	[−0.035, −0.011]	0/5	42–44
Entry/exit	TransferKAN − MaxViT-T	−0.010	[−0.020, −0.001]	0/5	42–44
MLSD	TransferKAN − Swin-B	−0.033	[−0.064, −0.0004]	1/5	42, 43
MLSD	TransferKAN − MaxViT-T	−0.030	[−0.059, +0.001]	1/5	42, 43

In entry/exit, TransferKAN is below Swin-B (ΔAUC = −0.023, [−0.035, −0.011]) and below MaxViT-T (−0.010, [−0.020, −0.001]), and loses in all five folds against both. In the distance task it is below Swin-B (−0.033, [−0.064, −0.0004], 1 of 5 folds). Against MaxViT-T the difference is similar (−0.030) but the interval includes zero. The claim that TransferKAN outperforms the transformer baselines is therefore not supported; the reverse holds for Swin-B in both tasks.

6.6 Wavelet-KAN Heads

Tables 6.7 and 6.8 show the wavelet-KAN heads next to the heads (a)–(d), Table 6.9 the pre-registered contrasts and Figure 6.5 the AUC against the number of head parameters.

Entry/exit. All heads, spline or wavelet, fall in the same AUC band (0.901–0.908). The first primary contrast, W2 against the MLP of the same size (b), is *inconclusive*: the AUC difference is small and inside the margin (−0.001, [−0.004, +0.002]), but the F1

Table 6.7: Entry/exit: wavelet-KAN heads against the heads (a)–(d) on the same ResNet-152 and recipe. Mean ± s.d. across folds (each fold averaged over the S training seeds); F1 is macro; AUC_{OOF} pools the out-of-fold predictions. Min: training minutes per fold (NVIDIA L4). Head ms: forward + backward of the head alone on a batch of 32 random 2048-d inputs.

Id	Head	Params	S	Acc	F1	AUC	AUC_{OOF}	Min	Head ms
(a)	Linear	4,098	3	0.846 ± 0.008	0.805 ± 0.012	0.904 ± 0.013	0.905	8.2	0.48
(b)	MLP-128	262,530	3	0.846 ± 0.011	0.806 ± 0.015	0.905 ± 0.014	0.906	8.3	0.73
(c)	MLP-iso	3,935,871	3	0.846 ± 0.009	0.807 ± 0.013	0.906 ± 0.014	0.907	8.4	0.73
(d)	Spline KAN = TransferKAN	3,936,000	3	0.846 ± 0.010	0.806 ± 0.013	0.905 ± 0.013	0.906	8.5	3.30
W1	Wav-KAN, Mexican hat, $h = 128$	1,049,860	3	0.835 ± 0.015	0.800 ± 0.019	0.902 ± 0.014	0.905	8.8	8.84
W2	Wav-KAN, Mexican hat, $h = 32$	262,468	3	0.834 ± 0.017	0.799 ± 0.022	0.902 ± 0.013	0.905	8.5	2.95
W3	Wav-KAN, DoG, $h = 128$	1,049,860	1	0.823 ± 0.020	0.793 ± 0.026	0.908 ± 0.015	0.907	8.7	7.60
W4	Wav-KAN, Morlet, $h = 128$	1,049,860	1	0.807 ± 0.023	0.779 ± 0.028	0.901 ± 0.015	0.900	8.7	8.55
W5	Wav-KAN, Mexican hat, $h = 480$	3,936,964	1	0.834 ± 0.017	0.799 ± 0.022	0.903 ± 0.013	0.902	10.6	38.36

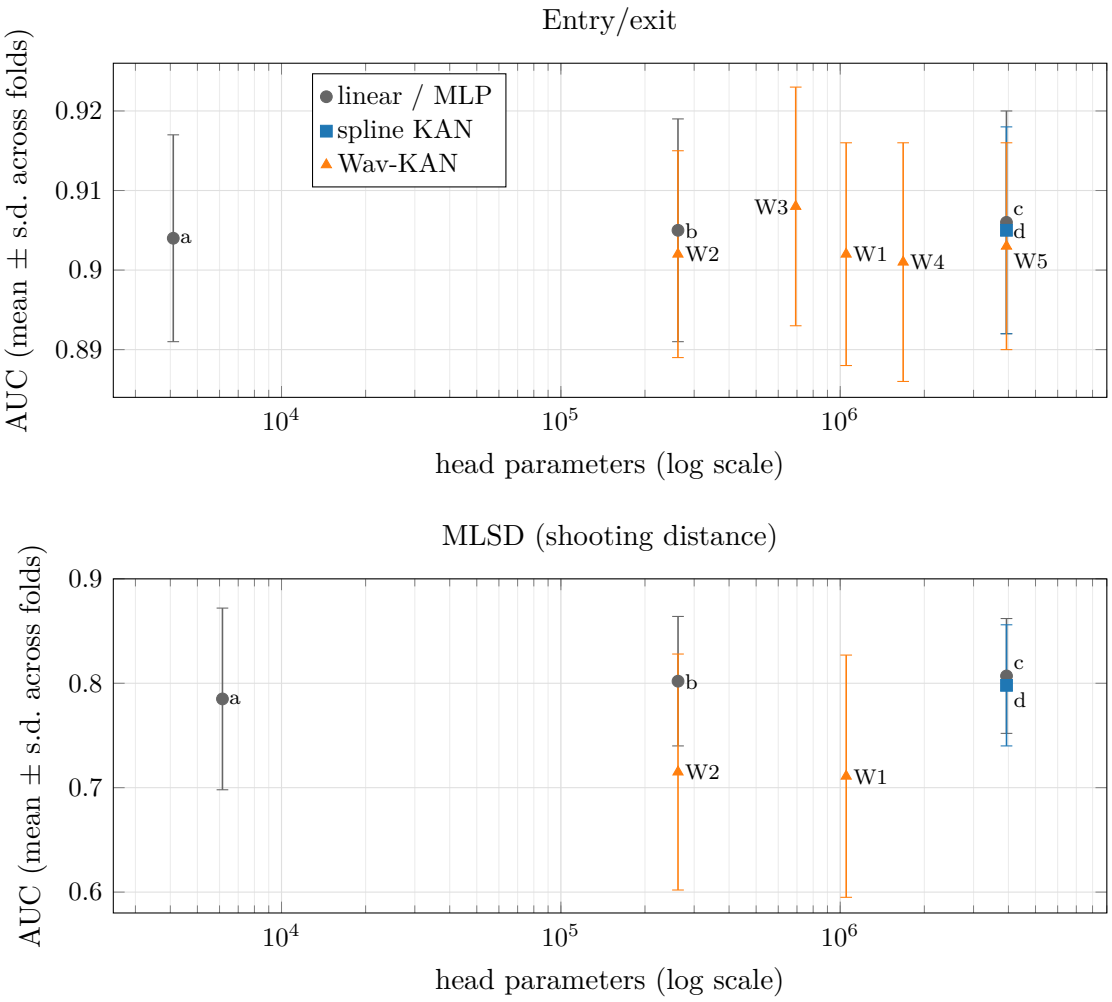


Figure 6.5: AUC (mean ± s.d. across folds) against the number of head parameters (log scale) for the linear and MLP heads, the spline KAN and the wavelet-KAN heads. W3 and W4 have the parameter count of W1 and are shifted horizontally only for legibility; W3–W5 use a single seed.

difference (-0.012, $[-0.023, -0.003]$) has its lower bound beyond -0.02, so non-inferiority is not shown; neither is inferiority, and W2 wins no fold. W1 and W5 are non-inferior to the matched MLP (c) and to the spline KAN (d) (W5 with a single seed), and W2 does not reach non-inferiority against (d) for the same F1 reason. The wavelet heads rank the images as well as the MLPs (same AUC) but, at the default threshold of 0.5, they have about one point less accuracy (0.834–0.835 against 0.846) and slightly lower F1 (0.799–0.800 against 0.805–0.807), with a slightly higher macro recall. This points to a shift of the decision threshold or of the calibration, not to a loss of separation. Among the single-seed sensitivity runs, the derivative-of-Gaussian head (W3) has the highest AUC of the table (0.908) but lower F1 and accuracy, and the Morlet head (W4) has the lowest F1 (ΔF1 against W1 $= -0.023$, $[-0.035, -0.012]$).

Table 6.8: MLSD (entries only; AUC = one-vs-rest macro): wavelet-KAN heads against the heads (a)–(d) on the same ResNet-152 and recipe. Mean ± s.d. across folds (each fold averaged over the S training seeds); F1 is macro; AUC_{OOF} pools the out-of-fold predictions. Min: training minutes per fold (NVIDIA L4). Head ms: forward + backward of the head alone on a batch of 32 random 2048-d inputs.

Id	Head	Params	S	Acc	F1	AUC	AUC_{OOF}	Min	Head ms
(a)	Linear	6,147	3	0.815 ± 0.014	0.488 ± 0.056	0.785 ± 0.087	0.797	6.3	0.48
(b)	MLP-128	262,659	3	0.813 ± 0.013	0.482 ± 0.050	0.802 ± 0.062	0.811	6.3	0.73
(c)	MLP-iso	3,937,791	3	0.821 ± 0.017	0.480 ± 0.050	0.807 ± 0.055	0.817	6.4	0.73
(d)	Spline KAN = TransferKAN	3,937,920	3	0.813 ± 0.017	0.477 ± 0.045	0.798 ± 0.058	0.805	6.5	3.30
W1	Wav-KAN, Mexican hat, $h = 128$	1,050,374	3	0.671 ± 0.043	0.418 ± 0.034	0.711 ± 0.116	0.730	6.7	8.84
W2	Wav-KAN, Mexican hat, $h = 32$	262,598	3	0.649 ± 0.039	0.410 ± 0.030	0.715 ± 0.113	0.740	6.4	2.95

Shooting distance. Here the wavelet heads lose clearly: AUC 0.711 ± 0.116 (W1) and 0.715 ± 0.113 (W2), against 0.785–0.807 for the other heads. Every contrast is inferior: W2 against the MLP (b) has $\Delta\text{AUC} = -0.071$ ($[-0.098, -0.043]$) and loses in all five folds, and the other contrasts range from -0.066 to -0.086. The variability across folds doubles (s.d. 0.11–0.12 against about 0.06) and accuracy falls to 0.65–0.67.

Cost and learned shapes. The wavelet heads are not faster. W2 takes almost the same time per fold as the MLP-128 (8.5 against 8.3 minutes) and the same as the spline KAN (8.46 against 8.49 minutes, a 0.35% difference, within the run-to-run noise of about 0.1 minute). W1 is 3.6% slower per fold than the spline KAN and 5.3% slower than the matched MLP, and W5 is 25–27% slower. On its own, the W1 head takes 8.8 ms for a forward and backward pass of 32 inputs and 223 MB of memory, against 0.73 ms and 0.2 MB for the matched MLP: each layer builds a tensor of size batch × outputs × inputs. The wavelets also barely changed shape. At the end of both phases and in both tasks, the mean scale stays between 0.99 and 1.003 (s.d. 0.013–0.020) and the mean translation stays close to 0 (s.d. 0.013–0.025), from initial values of 1 and 0. In practice the heads learned

Table 6.9: Pre-registered contrasts of the wavelet-KAN heads (Δ = W − reference) on the pooled out-of-fold predictions (probabilities averaged over the common seeds); 95% CI from a paired case-level bootstrap (2,000 resamples). Non-inferiority margin: −0.02 on the lower CI bound of both ΔAUC and ΔF1. Superior: higher AUC in at least 4/5 folds and ΔAUC CI above zero. Inferior: higher AUC in at most 1/5 folds and ΔAUC CI below zero. Inconclusive: none of the three. Wins: folds in which W has the higher AUC.

Task	Contrast	Role	Seeds	ΔAUC [95% CI]	ΔF1 [95% CI]	Wins	Outcome
E/S	W2 − (b)	primary 1	42–44	−0.001 [−0.004, +0.002]	−0.012 [−0.023, −0.003]	0/5	inconclusive
E/S	W1 − (c)	primary 2	42–44	−0.002 [−0.005, +0.002]	−0.003 [−0.014, +0.008]	0/5	non-inferior
E/S	W5 − (c)	primary 2	42	−0.003 [−0.008, +0.002]	−0.009 [−0.019, +0.003]	2/5	non-inferior
E/S	W1 − (d)	secondary	42–44	−0.001 [−0.004, +0.003]	−0.005 [−0.016, +0.005]	1/5	non-inferior
E/S	W2 − (d)	secondary	42–44	−0.001 [−0.003, +0.002]	−0.011 [−0.021, −0.002]	0/5	inconclusive
E/S	W5 − (d)	secondary	42	−0.002 [−0.007, +0.003]	−0.008 [−0.019, +0.002]	2/5	non-inferior
E/S	W1 − (b)	descriptive	42–44	−0.001 [−0.004, +0.003]	−0.007 [−0.019, +0.004]	1/5	non-inferior
E/S	W3 − W1	sensitivity	42	+0.002 [−0.002, +0.006]	−0.008 [−0.018, +0.001]	4/5	non-inferior
E/S	W4 − W1	sensitivity	42	−0.004 [−0.010, +0.001]	−0.023 [−0.035, −0.012]	1/5	inconclusive
E/S	W3 − (d)	sensitivity	42	+0.003 [−0.001, +0.007]	−0.014 [−0.027, −0.001]	4/5	inconclusive
E/S	W4 − (d)	sensitivity	42	−0.004 [−0.009, +0.001]	−0.028 [−0.044, −0.014]	1/5	inconclusive
MLSD	W2 − (b)	primary 1	42–44	−0.071 [−0.098, −0.043]	−0.069 [−0.117, −0.021]	0/5	inferior
MLSD	W1 − (c)	primary 2	42–44	−0.086 [−0.120, −0.052]	−0.070 [−0.121, −0.024]	0/5	inferior
MLSD	W1 − (d)	secondary	42–44	−0.075 [−0.105, −0.044]	−0.067 [−0.113, −0.024]	0/5	inferior
MLSD	W2 − (d)	secondary	42–44	−0.066 [−0.096, −0.038]	−0.070 [−0.119, −0.022]	0/5	inferior
MLSD	W1 − (b)	descriptive	42–44	−0.080 [−0.109, −0.050]	−0.066 [−0.112, −0.024]	0/5	inferior

the weights of fixed-shape wavelets, so no result can be attributed to the adaptation of the wavelet shape.

In short, a KAN with wavelets does not change the picture. In entry/exit it ties with the MLPs in AUC and is slightly worse at the default threshold; in the distance task it is clearly worse; and it is neither smaller in practice than an MLP of the same performance nor faster.

6.7 One Network for Both Tasks

Table 6.10 compares one multitask ResNet-152 with the pair of single-task networks, Table 6.11 gives the contrasts and Table 6.12 the cost.

Entry/exit. The multitask network is worse, but within the pre-registered margin. With KAN heads the AUC goes from 0.905 ± 0.013 to 0.894 ± 0.014 (ΔAUC = −0.011, $[-0.015, -0.006]$), and with MLP heads from 0.905 to 0.895 (−0.010, $[-0.014, -0.005]$). The multitask network loses in all five folds, so the rule classifies the result as worse, while the lower bound of the interval stays above the margin of −0.02. Both statements hold: the loss is small and consistent. The drop is larger at the default threshold: ΔF1 = −0.023 ($[-0.034, -0.011]$) with KAN heads and −0.022 ($[-0.034, -0.010]$) with MLP

Table 6.10: One multitask ResNet-152 (shared backbone, one head per task, $L = L_{\mathrm{E/S}} + L_{\mathrm{dist}}$ with exits masked out of L_{dist}) against the pair of single-task networks of the head ablation, same folds and recipe, seeds 42–44 on both sides. Mean ± s.d. across folds; AUC from probabilities (distance: one-vs-rest macro, entries only); F1 is macro; 4-class F1: exit/contact/close/distant pipeline (secondary, descriptive).

System	Nets	Params (M)	E/S AUC	E/S F1	Dist. AUC	Dist. F1	4-class F1
Two single-task nets, KAN heads (d)	2	124.16	0.905 ± 0.013	0.806 ± 0.013	0.798 ± 0.058	0.477 ± 0.045	0.500 ± 0.035
One multitask net, KAN heads	1	66.02	0.894 ± 0.014	0.784 ± 0.022	0.805 ± 0.065	0.482 ± 0.058	0.491 ± 0.045
Two single-task nets, MLP-128 heads (b)	2	116.81	0.905 ± 0.014	0.806 ± 0.015	0.802 ± 0.062	0.482 ± 0.050	0.506 ± 0.037
One multitask net, MLP-128 heads	1	58.67	0.895 ± 0.014	0.787 ± 0.017	0.806 ± 0.071	0.492 ± 0.063	0.499 ± 0.045

heads, beyond the margin, and accuracy falls from 0.846 to 0.820 and 0.823. In 5 of the 30 multitask runs the entry/exit accuracy of a fold fell slightly below 0.80 (0.792–0.797), which reflects this drop; the losses stayed finite and the AUC of those folds was normal (0.873–0.885).

Shooting distance. The multitask network is non-inferior. With KAN heads the AUC is 0.805 ± 0.065 against 0.798 ± 0.058 ($+0.007$, $[-0.003, +0.018]$, 3 of 5 folds), and with MLP heads 0.806 against 0.802 ($+0.004$, $[-0.006, +0.014]$). The macro F1 is also unchanged. The multitask accuracy is higher (0.824 against 0.813), but its balanced accuracy is equal or lower (0.511 against 0.515 with KAN heads, 0.520 for both with MLP heads), so the accuracy should not be read as a gain.

Table 6.11: Pre-registered contrasts of the multitask study (Δ = first − second; MT = multitask, ST = pair of single-task networks), on the pooled out-of-fold predictions (probabilities averaged over seeds 42–44); 95% CI from a paired case-level bootstrap (2,000 resamples). Non-inferiority margin: -0.02 on the lower CI bound of ΔAUC. Worse: higher AUC in at most 1/5 folds and ΔAUC CI below zero; "within margin": that CI is still above -0.02. Wins: folds in which the first system has the higher AUC (F1 for the 4-class rows).

Task	Contrast	Role	ΔAUC [95% CI]	ΔF1 [95% CI]	Wins	Outcome
E/S	MT-KAN − ST-KAN	primary	-0.011 $[-0.015, -0.006]$	-0.023 $[-0.034, -0.011]$	0/5	worse, within margin
E/S	MT-MLP − ST-MLP	primary	-0.010 $[-0.014, -0.005]$	-0.022 $[-0.034, -0.010]$	0/5	worse, within margin
E/S	MT-KAN − MT-MLP	secondary	-0.001 $[-0.003, +0.0004]$	-0.002 $[-0.008, +0.003]$	2/5	non-inferior
E/S	ST-KAN − ST-MLP	descriptive	-0.0001 $[-0.001, +0.001]$	-0.001 $[-0.006, +0.003]$	3/5	descriptive
Dist.	MT-KAN − ST-KAN	primary	$+0.007$ $[-0.003, +0.018]$	-0.002 $[-0.029, +0.027]$	3/5	non-inferior
Dist.	MT-MLP − ST-MLP	primary	$+0.004$ $[-0.006, +0.014]$	$+0.007$ $[-0.021, +0.035]$	3/5	non-inferior
Dist.	MT-KAN − MT-MLP	secondary	-0.002 $[-0.009, +0.003]$	-0.008 $[-0.014, -0.001]$	2/5	non-inferior
Dist.	ST-KAN − ST-MLP	descriptive	-0.005 $[-0.012, +0.003]$	$+0.001$ $[-0.007, +0.010]$	2/5	descriptive
4 classes	MT-KAN − ST-KAN	descriptive	—	-0.030 $[-0.061, -0.004]$	1/5	descriptive
4 classes	MT-MLP − ST-MLP	descriptive	—	-0.022 $[-0.052, +0.004]$	2/5	descriptive
4 classes	MT-KAN − MT-MLP	descriptive	—	-0.007 $[-0.013, -0.002]$	0/5	descriptive

KAN against MLP inside the multitask network. The two head types tie again: ΔAUC $= -0.001$ ($[-0.003, +0.0004]$) in entry/exit and -0.002 ($[-0.009, +0.003]$)

in distance. This compares head types, not efficiency: the two KAN heads have 7.87 million parameters, against 0.53 million for the two MLP heads.

Four-class pipeline (secondary, descriptive). Combining the two outputs into exit, contact, close and distant, the pooled macro F1 is 0.485 for the multitask network against 0.515 for the pair with KAN heads (ΔF1 = -0.030, $[-0.061, -0.004]$), and 0.492 against 0.514 with MLP heads (-0.022, $[-0.052, +0.004]$). The drop comes mostly from the entry/exit head: the multitask network sends more entries to "exit" (283 distant-shot entries predicted as exits, against 204 for the pair), which also raises the recall of exits (0.788 against 0.756). Contact recall in the pipeline goes from 5 to 2 of the 42 images, too few to support any conclusion about contact shots.

Table 6.12: Cost of obtaining both outputs on an NVIDIA L4 (AMP fp16). Training: minutes per fold (single-task = sum of the two networks; mean ± s.d. over the 15 units) and peak training memory. Inference: ms per image to obtain both outputs (one forward pass for the multitask network, two for the pair; 20 warm-up and 200 timed runs with CUDA events, mean ± s.d.) and peak inference memory at batch 32. Timing does not depend on the weights and was measured on freshly built models.

		Training		Inference (both outputs)		
System	Params (M)	min/fold	peak GB	ms/img, batch 1	ms/img, batch 32	peak MB
Two single-task nets, KAN heads (d)	124.16	14.9 ± 0.2	3.6	51.0 ± 2.1	2.22 ± 0.01	282
One multitask net, KAN heads	66.02	8.6 ± 0.1	3.7	27.3 ± 0.9	1.12 ± 0.01	163
Two single-task nets, MLP-128 heads (b)	116.81	14.6 ± 0.1	3.5	48.5 ± 1.0	2.22 ± 0.02	287
One multitask net, MLP-128 heads	58.67	8.1 ± 0.1	3.5	24.3 ± 1.3	1.11 ± 0.01	163

Cost. One network replaces two. With KAN heads the parameters fall from 124.16 to 66.02 million (-46.8%), and with MLP heads from 116.81 to 58.67 million (-49.8%). Training takes 8.6 instead of 14.9 minutes per fold (KAN) and 8.1 instead of 14.6 (MLP), with a standard deviation of 0.1–0.2 minute. Obtaining both outputs takes 1.12 instead of 2.22 ms per image at batch 32 (2.0 times faster) and 27.3 instead of 51.0 ms at batch 1. Peak inference memory at batch 32 falls from 282 to 163 MB, while peak training memory is about the same (3.7 against 3.6 GB). These savings follow directly from running one forward pass instead of two; they are a cost advantage of sharing the backbone, not a merit of a particular head.

6.8 Summary

Under one protocol without case leakage, the results support the following statements, all tied to a pre-registered rule:

- the hold-out of the public code of Lira et al. inflates AUC by +0.034 (entry/exit) and +0.155 to +0.158 (distance) with everything else fixed;

- the KAN head of TransferKAN is equivalent to linear and MLP heads in entry/exit (outcome B) and, in the distance task, below the matched MLP while equivalent to the MLP-128 and the linear head (outcome C);
- the best architecture is Swin-B, which outperforms TransferKAN in both tasks;
- wavelet-KAN heads do not change this picture: equal AUC in entry/exit, clearly worse in distance, and not faster;
- one multitask network costs about half of the pair of networks, is non-inferior in distance and slightly but consistently worse in entry/exit, within the margin.

The statements forbidden by the pre-registered rules remain unsupported: that changing only the head improves the results, that TransferKAN is a new state of the art, that it outperforms the transformer baselines, and that the KAN head is parameter-efficient.

Chapter 7

Conclusions

This dissertation set out to evaluate TransferKAN, a pre-trained ResNet-152 with a KAN classification head, for two forensic tasks on real crime-scene photographs: entry versus exit wounds and the medico-legal shooting distance. Re-examining an earlier comparison showed that its numbers could not be attributed to the KAN head. The work was therefore rebuilt around a single evaluation protocol without case leakage, with decision rules fixed before the experiments.

7.1 Answers to the Research Questions

RQ1: does the 2024 hold-out inflate the numbers? Yes. The hold-out of the public code of Lira et al. stratifies by case instead of grouping by case, so 98.7% (entry/exit) and 98.4% (distance) of its test images belong to cases seen in training. With the same network, recipe and seed, it raises the AUC from 0.906 to 0.940 in entry/exit (+0.034) and from 0.779–0.788 to 0.937–0.942 in shooting distance (+0.155 to +0.158). The effect is large where the task is hard and the classes are rare, and it also hides the variability across cases. Numbers obtained with this split are not comparable with case-grouped results.

RQ2: does the KAN head improve over linear and MLP heads? No. On the same ResNet-152 and recipe, the KAN head is equivalent to the linear and MLP heads in entry/exit (AUC 0.905 against 0.904–0.906; difference to the best MLP -0.001, 95% CI $[-0.002, +0.0003]$; outcome B). In shooting distance it is below the MLP with the same number of parameters (-0.011, $[-0.021, -0.003]$; outcome C) and equivalent to the MLP-128 and to the linear head. The KAN head of TransferKAN is not the source of its results, and with 3.94 million parameters it is not parameter-efficient.

RQ3: how do the backbones compare? With ImageNet pre-training and the same recipe, Swin-B is the best architecture in both tasks (AUC 0.929 in entry/exit and

0.838 in distance) and outperforms TransferKAN in both. TransferKAN sits in the middle of the twelve architectures, together with the ResNet-152 with a linear head. Compared with the published table of Lira et al., the other eleven backbones score higher than their published counterparts in entry/exit, while TransferKAN remains below their ResNet-152 in accuracy, F1 and AUC_c in both tasks. These differences come from the split, the pre-training and the recipe together, and not from the KAN head.

RQ4: does a wavelet KAN change the picture? No. In entry/exit the wavelet-KAN heads rank the images as well as the MLPs (equal AUC) but lose about one point of accuracy and a little F1 at the default threshold. The contrast with an MLP of the same size is inconclusive. In shooting distance they are clearly worse (AUC about 0.71 against 0.80). They are not faster, and their learned scales and translations barely moved from initialisation.

RQ5: can one network replace the pair? Partly. One ResNet-152 with two heads has 47–50% fewer parameters than the pair, trains in 8.1–8.6 instead of 14.6–14.9 minutes per fold and obtains both outputs twice as fast. It is non-inferior in shooting distance and worse in entry/exit by about 0.01 of AUC in every fold, which is within the pre-registered margin of 0.02 but consistent, with a larger drop in F1. When the cost matters more than one hundredth of AUC in entry/exit, it is a reasonable replacement.

Taken together, the hypothesis that motivated this work, that replacing the MLP head with a KAN improves the classification, is answered negatively and with two KAN families. The positive contributions are methodological, an audited split and a protocol under which the architectures can be compared, and practical, a single network for both tasks at about half the cost.

7.2 Limitations

- **One database.** All images come from one police force in one region; there was no external validation.
- **Contact shots.** With 42 images from 16 cases, only 13–17% of the contact images are recognised when their case is not in the training set, and nothing specific can be claimed about this class.
- **Seeds.** Swin-B and MaxViT-T have two seeds in the distance task, and most other backbones, the small KAN (e) and the wavelet heads W3–W5 have a single seed.
- **The split of Lira et al.** The leakage experiment reproduces the method of their public code, not the exact partition of their published tables, which cannot be recovered.

- **Last epoch without validation.** Every model is evaluated at the last epoch, without an inner validation split. This is fair across models but not tuned for any of them.
- **Recipe.** The recipe of the original TransferKAN notebook was kept for every model, including a strong Gaussian noise that may erase fine traces such as stippling. Other recipes might change absolute numbers.
- **Pre-training not isolated.** The comparison with the published table suggests a large effect of ImageNet pre-training, but no model was trained from scratch under the single protocol, so that effect is not measured here.

7.3 Future Work

- **KAN on the best backbone.** Test a KAN head on Swin-B, the best backbone, to check whether the head ablation holds on a stronger feature extractor.
- **More contact cases and external data.** Collect more contact cases and images from other institutions, so that the minority class can be learned and the models validated outside the original database.
- **Calibration and thresholds.** Calibrate the probabilities and choose the decision threshold on an inner validation split; the wavelet heads and the multitask network lose mainly at the default threshold.
- **Better multitask training.** Weight the task losses, for example by their uncertainty, to recover the small loss of the multitask network in entry/exit.
- **The effect of pre-training.** Train some backbones from scratch under the single protocol to measure the effect of pre-training, the factor that the comparison with the published numbers could not isolate.

References

[1] United Nations Office on Drugs and Crime (UNODC): *Global Study on Homicide 2023*. `https://www.unodc.org/unodc/en/data-and-analysis/global-study-on-homicide.html`, 2023. Accessed: 2024-03-08. 1

[2] Queiroz Nogueira Lira, Renato, Luana Geovana Motta de Sousa, Maisa Luana Memoria Pinho, Renan Cesar Pinto da Silva Andrade de Lima, Pedro Garcia Freitas, Bruno Scholles Soares Dias, Andreia Cristina Breda de Souza, and André Ferreira Leite: *Deep learning-based human gunshot wounds classification*. International Journal of Legal Medicine, 139(2):651–666, 2025. xi, xii, 1, 9, 10, 13, 15, 17, 27, 31, 46, 47

[3] Liu, Ziming, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark: *KAN: Kolmogorov-Arnold Networks*. arXiv preprint arXiv:2404.19756, 2024. 1, 6, 10

[4] Di Maio, Vincent J M: *Gunshot Wounds: Practical Aspects of Firearms, Ballistics, and Forensic Techniques*. CRC Press, Boca Raton, FL, USA, 3rd edition, 2015. 4

[5] Saukko, Pekka J and Bernard Knight: *Knight's Forensic Pathology*. CRC Press, Boca Raton, FL, USA, 4th edition, 2016. 4

[6] He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun: *Deep residual learning for image recognition*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. 5, 12, 19

[7] Deng, Jia, Wei Dong, Richard Socher, Li Jia Li, Kai Li, and Li Fei-Fei: *ImageNet: A large-scale hierarchical image database*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009. 5, 12, 18

[8] Liu, Ze, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo: *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, 2021. 5, 19

[9] Tu, Zhengzhong, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan Bovik, and Yinxiao Li: *MaxViT: Multi-axis Vision Transformer*. In *Computer Vision – ECCV 2022*, volume 13684 of *Lecture Notes in Computer Science*, pages 459–479. Springer, 2022. 5, 19

[10] Blealtan: *efficient-kan: An efficient implementation of Kolmogorov-Arnold Network*. `https://github.com/Blealtan/efficient-kan`, 2024. Commit 7b6ce1c. 6, 11, 12, 19

[11] Bozorgasl, Zavareh and Hao Chen: *Wav-KAN: Wavelet Kolmogorov-Arnold Networks*. arXiv preprint arXiv:2405.12832, 2024. 6, 11, 13, 20

[12] Kapoor, Sayash and Arvind Narayanan: *Leakage and the reproducibility crisis in machine-learning-based science*. Patterns, 4(9):100804, 2023. 7

[13] Roberts, David R., Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann: *Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure*. Ecography, 40(8):913–929, 2017. 7

[14] Sechidis, Konstantinos, Grigorios Tsoumakas, and Ioannis Vlahavas: *On the Stratification of Multi-label Data*. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2011)*, volume 6913 of *Lecture Notes in Computer Science*, pages 145–158. Springer, 2011. 7, 10, 20

[15] Fawcett, Tom: *An introduction to ROC analysis*. Pattern Recognition Letters, 27(8):861–874, 2006. 7, 21

[16] Caruana, Rich: *Multitask Learning*. Machine Learning, 28(1):41–75, 1997. 8, 21

[17] Ruder, Sebastian: *An Overview of Multi-Task Learning in Deep Neural Networks*. arXiv preprint arXiv:1706.05098, 2017. 8, 21

[18] Efron, Bradley and Robert J. Tibshirani: *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993. 8, 22

[19] Oura, Petteri, Anneli Junno, and Juho Antti Junno: *Deep learning in forensic gunshot wound interpretation—a proof-of-concept study*. International Journal of Legal Medicine, 135:2101–2106, 2021. 9

[20] Cheng, Jerome, Carl Schmidt, Allecia Wilson, Zhaohui Wang, Wei Hao, Joshua Pantanowitz, Christopher Morris, Robert Tashjian, and Liron Pantanowitz: *Artificial intelligence for human gunshot wound classification*. Journal of Pathology Informatics, 15:100361, 2024. 9

[21] LISA/UnB: *LeGuWoI: Learned Gunshot Wound Identification (source code)*. `https://gitlab.com/lisa-unb/leguwoi`, 2025. Commit 83d82f1 (2025-04-03). 10, 16, 19, 20, 22

[22] Drokin, Ivan: *torch-conv-kan: Kolmogorov-Arnold convolutional networks in PyTorch*. `https://github.com/IvanDrokin/torch-conv-kan`, 2024. Commit b412a3b. 11, 20, 48

[23] Garcia Freitas, Pedro: *FDCPUnBGunshotDB: gunshot wound image database*. `https://github.com/pedrogarciafreitas/FDCPUnBGunshotDB`, 2025. Commit f679389. 15

[24] Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay: *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12:2825–2830, 2011. 16

[25] Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, *et al.*: *PyTorch: An imperative style, high-performance deep learning library.* In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019. 18

[26] Loshchilov, Ilya and Frank Hutter: *Decoupled Weight Decay Regularization.* In *International Conference on Learning Representations (ICLR)*, 2019. 18

[27] Loshchilov, Ilya and Frank Hutter: *SGDR: Stochastic Gradient Descent with Warm Restarts.* In *International Conference on Learning Representations (ICLR)*, 2017. 18

[28] Buslaev, Alexander, Vladimir I Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A Kalinin: *Albumentations: Fast and flexible image augmentations.* Information, 11(2):125, 2020. 18

[29] Micikevicius, Paulius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu: *Mixed Precision Training.* In *International Conference on Learning Representations (ICLR)*, 2018. 19

[30] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton: *ImageNet Classification with Deep Convolutional Neural Networks.* In *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, pages 1097–1105, 2012. 19

[31] Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang Chieh Chen: *MobileNetV2: Inverted Residuals and Linear Bottlenecks.* In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. 19

[32] Tan, Mingxing, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le: *MnasNet: Platform-Aware Neural Architecture Search for Mobile.* In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2820–2828, 2019. 19

[33] Ma, Ningning, Xiangyu Zhang, Hai Tao Zheng, and Jian Sun: *ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design.* In *Computer Vision – ECCV 2018*, volume 11218 of *Lecture Notes in Computer Science*, pages 116–131. Springer, 2018. 19

[34] Iandola, Forrest N., Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer: *SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size.* arXiv preprint arXiv:1602.07360, 2016. 19

[35] Huang, Gao, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger: *Densely Connected Convolutional Networks*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017. 19

[36] Tan, Mingxing and Quoc V. Le: *EfficientNetV2: Smaller Models and Faster Training*. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 10096–10106, 2021. 19

[37] Simonyan, Karen and Andrew Zisserman: *Very Deep Convolutional Networks for Large-Scale Image Recognition*. In *International Conference on Learning Representations (ICLR)*, 2015. 19

[38] Radosavovic, Ilija, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár: *Designing Network Design Spaces*. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10428–10436, 2020. 19

Appendix A

Results in the Format of Lira et al.

Lira et al. [2] report each metric with macro (M), micro (m) and weighted (W) averages. For readers who want to compare formats, Tables A.1 and A.2 give the results of the single protocol in that layout. In the binary task, micro precision, recall and F1 equal the accuracy and macro specificity equals macro recall, which is why the main tables omit them. Here the AUC is computed from probabilities; the AUC from predicted classes used by Lira et al. is in Tables 6.4 and 6.5. The comparison with their numbers is subject to the caveats of Section 6.4.

Table A.1: Entry/exit under the single protocol, in the format of Lira et al. [2]: M = macro, m = micro, W = weighted. Mean across folds (the standard deviations are in the result files of the repository); AUC from predicted probabilities. Seeds as in the main tables.

	Acc	Precision			Recall			F1			AUC			Specificity		
Model		M	m	W	M	m	W	M	m	W	M	m	W	M	m	W
ResNet152 + Linear (a)	0.846	0.800	0.846	0.852	0.814	0.846	0.846	0.805	0.846	0.847	0.904	0.925	0.904	0.814	0.846	0.783
ResNet152 + MLP-128 (b)	0.846	0.800	0.846	0.852	0.815	0.846	0.846	0.806	0.846	0.848	0.905	0.926	0.905	0.815	0.846	0.784
ResNet152 + MLP-iso (c)	0.846	0.801	0.846	0.853	0.817	0.846	0.846	0.807	0.846	0.848	0.906	0.926	0.906	0.817	0.846	0.789
ResNet152 + KAN (d) = TransferKAN	0.846	0.800	0.846	0.852	0.816	0.846	0.846	0.806	0.846	0.848	0.905	0.925	0.905	0.816	0.846	0.786
ResNet152 + KAN-8 (e)	0.846	0.800	0.846	0.854	0.819	0.846	0.846	0.808	0.846	0.848	0.903	0.921	0.903	0.819	0.846	0.793
AlexNet	0.803	0.750	0.803	0.822	0.782	0.803	0.803	0.761	0.803	0.809	0.859	0.881	0.859	0.782	0.803	0.761
MobileNetV2	0.812	0.763	0.812	0.835	0.802	0.812	0.812	0.776	0.812	0.819	0.877	0.890	0.877	0.802	0.812	0.792
MNASNet1.3	0.837	0.829	0.837	0.835	0.725	0.837	0.837	0.753	0.837	0.822	0.872	0.911	0.872	0.725	0.837	0.613
ShuffleNetV2 x0.5	0.746	0.706	0.746	0.799	0.755	0.746	0.746	0.713	0.746	0.759	0.839	0.826	0.839	0.755	0.746	0.764
SqueezeNet1.0	0.798	0.749	0.798	0.824	0.787	0.798	0.798	0.760	0.798	0.805	0.866	0.875	0.866	0.787	0.798	0.776
DenseNet169	0.858	0.814	0.858	0.863	0.830	0.858	0.858	0.821	0.858	0.860	0.904	0.924	0.904	0.830	0.858	0.801
MaxViT-T	0.864	0.821	0.864	0.873	0.846	0.864	0.864	0.831	0.864	0.867	0.917	0.930	0.917	0.846	0.864	0.827
Swin-B	0.884	0.850	0.884	0.884	0.850	0.884	0.884	0.850	0.884	0.884	0.929	0.948	0.929	0.850	0.884	0.816
EfficientNetV2-L	0.868	0.828	0.868	0.870	0.836	0.868	0.868	0.831	0.868	0.869	0.916	0.934	0.916	0.836	0.868	0.804
VGG16	0.841	0.794	0.841	0.842	0.796	0.841	0.841	0.794	0.841	0.841	0.891	0.920	0.891	0.796	0.841	0.752
RegNetY-16GF	0.852	0.807	0.852	0.858	0.824	0.852	0.852	0.815	0.852	0.854	0.906	0.925	0.906	0.824	0.852	0.796

Table A.2: MLSD (entries only) under the single protocol, in the format of Lira et al. [2]: M = macro, m = micro, W = weighted. Mean across folds (the standard deviations are in the result files of the repository); AUC from predicted probabilities. Seeds as in the main tables.

	Acc	Precision			Recall			F1			AUC			Specificity		
Model		M	m	W	M	m	W	M	m	W	M	m	W	M	m	W
ResNet152 + Linear (a)	0.815	0.487	0.815	0.843	0.524	0.815	0.815	0.488	0.815	0.825	0.785	0.937	0.820	0.816	0.908	0.632
ResNet152 + MLP-128 (b)	0.813	0.477	0.813	0.842	0.520	0.813	0.813	0.482	0.813	0.823	0.802	0.938	0.827	0.817	0.906	0.639
ResNet152 + MLP-iso (c)	0.821	0.482	0.821	0.842	0.510	0.821	0.821	0.480	0.821	0.828	0.807	0.941	0.824	0.814	0.910	0.622
ResNet152 + KAN (d) = TransferKAN	0.813	0.477	0.813	0.840	0.515	0.813	0.813	0.477	0.813	0.822	0.798	0.937	0.822	0.813	0.906	0.628
AlexNet	0.780	0.485	0.780	0.844	0.561	0.780	0.780	0.498	0.780	0.804	0.806	0.923	0.823	0.827	0.890	0.700
MobileNetV2	0.778	0.457	0.778	0.834	0.513	0.778	0.778	0.464	0.778	0.800	0.806	0.923	0.806	0.807	0.889	0.643
MNASNet1.3	0.451	0.383	0.451	0.841	0.455	0.451	0.451	0.294	0.451	0.518	0.710	0.678	0.768	0.765	0.725	0.843
ShuffleNetV2 x0.5	0.854	0.483	0.854	0.832	0.474	0.854	0.854	0.474	0.854	0.840	0.776	0.939	0.770	0.780	0.927	0.487
SqueezeNet1.0	0.788	0.487	0.788	0.837	0.525	0.788	0.788	0.490	0.788	0.807	0.805	0.924	0.803	0.807	0.894	0.632
DenseNet169	0.820	0.505	0.820	0.848	0.556	0.820	0.820	0.519	0.820	0.831	0.821	0.942	0.839	0.826	0.910	0.658
MaxViT-T	0.810	0.498	0.810	0.851	0.555	0.810	0.810	0.511	0.810	0.826	0.828	0.940	0.833	0.828	0.905	0.673
Swin-B	0.825	0.504	0.825	0.850	0.540	0.825	0.825	0.505	0.825	0.834	0.838	0.944	0.843	0.829	0.913	0.661
EfficientNetV2-L	0.830	0.499	0.830	0.847	0.529	0.830	0.830	0.495	0.830	0.835	0.744	0.936	0.800	0.817	0.915	0.621
VGG16	0.844	0.469	0.844	0.839	0.510	0.844	0.844	0.481	0.844	0.840	0.776	0.944	0.782	0.802	0.922	0.563
RegNetY-16GF	0.826	0.494	0.826	0.849	0.531	0.826	0.826	0.496	0.826	0.834	0.773	0.941	0.826	0.826	0.913	0.652

Appendix B

Exploratory Experiments

Before the single protocol, three KAN architectures trained from scratch were explored to see how far a KAN could go without a pre-trained feature extractor. They are recorded here for completeness. Their numbers come from a different setting and are *not* comparable with any result of Chapter 6.

- **ResKANet-18:** a residual network whose convolutions are themselves parameterised by splines (`reskanet_18x32p` from `torch-conv-kan` [22]), trained from scratch with 128×128 inputs.
- **PureConvKAN:** three convolutional KAN blocks (16, 32 and 64 channels), each followed by max-pooling and batch normalisation, then global average pooling, Dropout(0.5) and a linear output layer; trained from scratch with 128×128 inputs.
- **PureKAN:** the image flattened into a vector of pixels and fed directly to a KAN, with no convolution; 64×64 inputs ($3 \times 64 \times 64 = 12{,}288$ inputs).

ResKANet-18 was trained only for entry/exit, and PureConvKAN and PureKAN for both tasks. All were trained for 60 epochs end to end with AdamW and batches of 8 images. They were evaluated on a single 80/20 hold-out split separated by case, which avoids case leakage but is a single split, at lower resolution and with a recipe different from the single protocol. Table B.1 gives their results.

The only reading these numbers support is qualitative: without a pre-trained extractor, the KAN architectures reach 0.69–0.82 accuracy in entry/exit, and in the distance task the macro AUC of the two is close to or below chance (0.46–0.54), that is, they learn little about the minority classes. This is consistent with the main finding that the pre-trained backbone, and not the KAN, carries the discrimination.

Table B.1: Exploratory KAN architectures trained from scratch (no ImageNet pre-training) in an earlier phase of this work. They used a single 80/20 hold-out split by case, inputs of 64 or 128 px, batches of 8 and their own recipe, so they are *not* comparable with any number obtained under the single protocol. M = macro, m = micro, W = weighted.

		Acc	Precision			Recall			F1			AUC			Specificity		
Model	Input		M	m	W	M	m	W	M	m	W	M	m	W	M	m	W
Entry/exit																	
ResKANet-18	128 px	0.822	0.767	0.822	0.820	0.762	0.822	0.822	0.765	0.822	0.821	0.839	0.885	0.839	0.762	0.822	0.703
PureConvKAN	128 px	0.707	0.614	0.707	0.704	0.611	0.707	0.707	0.613	0.707	0.706	0.682	0.768	0.682	0.611	0.707	0.516
PureKAN	64 px	0.692	0.614	0.692	0.692	0.614	0.692	0.692	0.614	0.692	0.692	0.684	0.733	0.684	0.614	0.692	0.536
MLSD																	
PureConvKAN	128 px	0.803	0.268	0.803	0.645	0.333	0.803	0.803	0.297	0.803	0.716	0.536	0.888	0.509	0.667	0.902	0.197
PureKAN	64 px	0.712	0.355	0.712	0.689	0.324	0.712	0.712	0.328	0.712	0.693	0.463	0.841	0.496	0.679	0.856	0.326